Università Politecnica delle Marche

Ingegneria Informatica e dell'Automazione

Rilevamento usura pala a partire dall'analisi del log di volo di un drone esarotore



Corso di

Manutenzione preventiva

Anno accademico 2025-2026

Studenti:

Ettore Ricci

Nicola Sebastianelli

Professore:

Alessandro Freddi

Indice

Introduzione2

1 Introduzione **3**
- 1.1 Descrizione del progetto . . . 3
- 1.2 Descrizioni delle variabili del datalog . . . 3

2 Creazione Tabella di timetable **7**
- 2.1 Caricamento e Sincronizzazione dei Segnali . . . 7
- 2.2 Costruzione della Struttura Dati (dataTable) . . . 7
- 2.3 Etichettatura delle Classi (Fault Codes) . . . 8

3 Estrazione feature dalla timetable **9**
- 3.1 Impostazione della Frame Policy . . . 9
- 3.2 Processo di Estrazione delle Feature . . . 9
- 3.3 Feature nel dominio del tempo . . . 10
- 3.4 Feature nel dominio della frequenza . . . 11
  - 3.4.1 Configurazione dell'estrazione dei Picchi (Spectral Peaks) . . . 12
- 3.5 Migliori Feature . . . 12
  - 3.5.1 Ranking tramite One-Way ANOVA . . . 13
  - 3.5.2 Ranking tramite Kruskal-Wallis . . . 13
  - 3.5.3 Definizione dei Set di Feature per il Confronto . . . 14

4 Classificazione dei guasti **15**
- 4.1 Divisione del Dataset . . . 15
- 4.2 Metriche di Valutazione . . . 15
- 4.3 Classificatori Utilizzati . . . 16
- 4.4 Alberi decisionali . . . 17
  - 4.4.1 Due classi . . . 17
  - 4.4.2 Tre classi . . . 19
- 4.5 K-Nearest Neighbors (KNN) . . . 20
  - 4.5.1 Due classi . . . 21
  - 4.5.2 Tre classi . . . 22
- 4.6 Support Vector Machine (SVM) . . . 24
  - 4.6.1 Due classi . . . 24
  - 4.6.2 Tre classi . . . 26
- 4.7 Random Forest . . . 28
  - 4.7.1 Due classi . . . 28
  - 4.7.2 Tre classi . . . 30
- 4.8 Reti Neurali . . . 32

4.8.1 Due classi 32
4.8.2 Tre classi 34

5 Risultati **36**
5.1 Analisi per Due Classi 36
5.2 Analisi per Tre Classi 37
5.3 Confronto tra ANOVA e Kruskal-Wallis 37

6 Explainability **38**
6.1 Decision Tree: Interpretazione Diretta e Rule Extraction 39
6.2 Reti Neurali (NN): Modello Surrogato 42
6.3 Random Forest: OOB Importance e Analisi Locale (Gini Decrease) 46
6.4 SVM: Permutation Feature Importance 54
6.5 K-Nearest Neighbors (KNN): Case Study Locale e Analisi dei Vicini 56

7 Conclusioni **59**

1 Introduzione

1.1 Descrizione del progetto

Il presente progetto si inserisce nel contesto dell'analisi dei dati di volo di un drone esarotore, acquisiti mediante campagne di misura condotte presso il laboratorio LAIR dell'Università Politecnica delle Marche. L'obiettivo principale è quello di effettuare un'analisi approfondita dei dataset relativi al volo del drone, con particolare attenzione alle differenze di comportamento tra pale nuove e pale soggette a usura, appositamente danneggiate, al fine di discriminarne il funzionamento.
La rotazione ad alta velocità delle eliche del drone rende infatti ogni minimo contatto potenzialmente dannoso, causando deterioramenti significativi del bordo della pala o, nei casi più gravi, la frattura della sua estremità. Questo studio si propone pertanto di integrare le diverse tipologie di danneggiamento delle pale, derivanti sia da contatti accidentali sia da fenomeni di usura, con l'obiettivo di sviluppare un modello predittivo in grado di distinguere efficacemente il comportamento delle pale nuove da quello delle pale danneggiate, ponendo particolare attenzione anche all'interpretabilità (*explainability*) del modello stesso, così da comprendere le motivazioni alla base delle decisioni di classificazione.
Il dataset analizzato comprende diverse sessioni di volo, ciascuna caratterizzata da specifiche condizioni operative: pala nuova, pala leggermente usurata (con tagli pari al 5% della lunghezza totale) e pala gravemente usurata (con tagli pari al 10% della lunghezza totale).
Il drone utilizzato presenta una configurazione hardware e software dedicata a questo tipo di analisi, composta dai seguenti elementi:

- frame DJI F550;
- microcontrollore Pixhawk con stack ArduPilot;
- firmware modificato per l'acquisizione selettiva dei dati ad alta frequenza.

Per il trattamento dei dati, l'estrazione delle feature e la fase di classificazione verranno utilizzati MATLAB e due strumenti interattivi messi a disposizione dall'ambiente di sviluppo: il *Diagnostic Feature Designer (DFD)* e il *Classification Learner*.

1.2 Descrizioni delle variabili del datalog

L'analisi preliminare è stata condotta su un insieme di 18 dataset di volo, suddivisi in tre categorie principali in base allo stato di salute delle pale del drone:

- **NO FAULT**: 6 file relativi a voli in condizioni nominali (nessun guasto).

- **FAULT M 5%**: 6 file relativi a voli con un guasto di lieve entità (taglio del 5% sulla pala) su ciascuno dei 6 motori (da M1 a M6).
- **FAULT M 10%**: 6 file relativi a voli con un guasto di entità maggiore (taglio del 10% sulla pala) su ciascuno dei 6 motori.

Durante la fase di estrazione dati, sono state selezionate le variabili più significative per la caratterizzazione dinamica del drone, provenienti da diverse strutture del log di ArduPilot ('IMU', 'PWM', 'ESC', 'ATTITUDE', 'XKF1', 'VIBE').

Di seguito si riporta l'elenco delle variabili utilizzate nel progetto:

- **IMU (Inertial Measurement Unit)**: fornisce i dati grezzi dei sensori inerziali, fondamentali per analizzare le vibrazioni e la risposta dinamica del veicolo. I segnali sono campionati nativamente ad alta frequenza (fino a 350 Hz).
  - `acc_x`: Accelerazione lineare misurata lungo l'asse X [m/s^2].
  - `acc_y`: Accelerazione lineare misurata lungo l'asse Y [m/s^2].
  - `acc_z`: Accelerazione lineare misurata lungo l'asse Z [m/s^2].
  - `gyr_x`: Velocità angolare misurata attorno all'asse X [rad/s].
  - `gyr_y`: Velocità angolare misurata attorno all'asse Y [rad/s].
  - `gyr_z`: Velocità angolare misurata attorno all'asse Z [rad/s].
- **PWM (Pulse Width Modulation)**: rappresenta i comandi inviati dal flight controller ai motori. Questi segnali riflettono lo sforzo correttivo del controllo automatico.
  - `pwm_1`: Segnale di comando per il Motore 1.
  - `pwm_2`: Segnale di comando per il Motore 2.
  - `pwm_3`: Segnale di comando per il Motore 3.
  - `pwm_4`: Segnale di comando per il Motore 4.
  - `pwm_5`: Segnale di comando per il Motore 5.
  - `pwm_6`: Segnale di comando per il Motore 6.
- **ESC (Electronic Speed Controller)**: fornisce il feedback di telemetria proveniente dai regolatori di velocità dei motori.
  - `esc_1`: Velocità di rotazione del Motore 1 [RPM].

  - `esc_2`: Velocità di rotazione del Motore 2 [RPM].
  - `esc_3`: Velocità di rotazione del Motore 3 [RPM].
  - `esc_4`: Velocità di rotazione del Motore 4 [RPM].
  - `esc_5`: Velocità di rotazione del Motore 5 [RPM].
  - `esc_6`: Velocità di rotazione del Motore 6 [RPM].

  - `cur_1`: Corrente assorbita dal Motore 1 [A].
  - `cur_2`: Corrente assorbita dal Motore 2 [A].
  - `cur_3`: Corrente assorbita dal Motore 3 [A].
  - `cur_4`: Corrente assorbita dal Motore 4 [A].
  - `cur_5`: Corrente assorbita dal Motore 5 [A].
  - `cur_6`: Corrente assorbita dal Motore 6 [A].

- **ATTITUDE**: contiene le stime degli angoli di Eulero (reali e desiderati) fornite dal filtro di navigazione.
  - `roll`: Angolo di Rollio effettivo (rotazione asse X) [deg].
  - `pitch`: Angolo di Beccheggio effettivo (rotazione asse Y) [deg].
  - `yaw`: Angolo di Imbardata effettivo (rotazione asse Z) [deg].
  - `roll_des`: Angolo di Rollio desiderato (Setpoint) [deg].
  - `pitch_des`: Angolo di Beccheggio desiderato (Setpoint) [deg].
  - `yaw_des`: Angolo di Imbardata desiderato (Setpoint) [deg].
- **XKF1 (Extended Kalman Filter)**: fornisce le stime di velocità lineare nel sistema di riferimento locale NED (North-East-Down).
  - `vn`: Velocità lineare stimata lungo l'asse Nord [m/s].
  - `ve`: Velocità lineare stimata lungo l'asse Est [m/s].
  - `vd`: Velocità lineare stimata lungo l'asse Verticale (down) [m/s].
- **VIBE**: fornisce metriche specifiche sulle vibrazioni, spesso pre-filtrate o elaborate dal firmware per monitorare le vibrazioni.
  - `vibe_x`: Livello di vibrazione medio lungo l'asse X.
  - `vibe_y`: Livello di vibrazione medio lungo l'asse Y.
  - `vibe_z`: Livello di vibrazione medio lungo l'asse Z.

Tutte le variabili elencate sono acquisite con frequenze di campionamento differenti e non sincronizzate. Come evidenziato nel codice di estrazione:

```
% Desired sync rate
Fs = 350;
syncmethod="zero_order_hold";
```

È stata scelta una frequenza di campionamento comune $F_s = 350$ Hz (corrispondente alla frequenza massima dei sensori IMU) per uniformare il dataset. La sincronizzazione è avvenuta tramite metodo *Zero Order Hold* (mantenimento dell'ultimo valore valido), garantendo l'allineamento temporale di tutti i segnali in un'unica `timetable` per ciascun volo.

2 Creazione Tabella di timetable

La fase di pre-processing è stata necessaria per trasformare i dati grezzi, acquisiti dai sensori eterogenei del drone, in una struttura dati coerente e adatta all'estrazione delle feature. I log originali di ArduPilot, infatti, presentano frequenze di campionamento variabili a seconda del sensore (es. l'IMU campiona ad alta frequenza, mentre i dati GPS o batteria hanno frequenze molto più basse).

Il processo di creazione della `timetable` è stato implementato in MATLAB seguendo quattro step fondamentali, descritti di seguito.

2.1 Caricamento e Sincronizzazione dei Segnali

Lo script di elaborazione itera su una lista di 18 file di log (6 per la condizione *No Fault*, 6 per *Fault 5%* e 6 per *Fault 10%*). Poiché i vettori temporali dei vari sensori non sono allineati, è stato utilizzato il comando MATLAB `synchronize` per ricampionare tutti i segnali su una base temporale comune.

Le specifiche di sincronizzazione adottate sono le seguenti:

- **Frequenza di Campionamento (F_s)**: È stata impostata a **350 Hz**. Tale valore è stato scelto per preservare l'integrità informativa dei segnali inerziali (IMU), che costituiscono la fonte dati a più alta frequenza e maggior contenuto informativo per l'analisi vibrazionale.
- **Metodo di Interpolazione**: È stato scelto il metodo `zero_order_hold` (mantenimento di ordine zero). A differenza dell'interpolazione lineare, questo metodo mantiene l'ultimo valore valido noto fino all'arrivo del campione successivo. Questa scelta è ideale per segnali digitali o discreti (come i comandi PWM o gli stati del sistema) e per evitare l'introduzione di artefatti artificiali nei segnali vibrazionali grezzi.

2.2 Costruzione della Struttura Dati (dataTable)

Per ogni volo, i segnali sincronizzati sono stati convertiti in oggetti `timetable` tramite la funzione `array2timetable`. Successivamente, è stata costruita una tabella master (`dataTable`) di dimensioni $18 \times N$ (dove N è il numero di variabili estratte), strutturata come segue:

- Ogni **riga** rappresenta un singolo volo (file di log).
- Ogni **colonna** contiene una cella con la `timetable` del segnale specifico (es. `acc_x`, `esc_1`, ecc.) relativa all'intera durata del volo.

Questa struttura nidificata permette di mantenere l'associazione tra i diversi sensori dello stesso volo, facilitando le successive operazioni di estrazione delle feature basate su finestre temporali.

2.3 Etichettatura delle Classi (Fault Codes)

Al termine della strutturazione dei dati, è stata aggiunta alla tabella una colonna target denominata `faultCode`, fondamentale per l'addestramento supervisionato dei modelli di classificazione. Sulla base dei nomi dei file caricati, sono state assegnate le seguenti etichette numeriche categoriche:

- **Classe 0**: Voli nominali (NO_FAULT).
- **Classe 5**: Voli con usura lieve ($FAULT_M_5$), indicante un danno del 5% sulla pala.
- **Classe 10**: Voli con usura severa ($FAULT_M_10$), indicante un danno del 10% sulla pala.

La tabella finale `dataTable` costituisce l'input diretto per il *Diagnostic Feature Designer*, modulo utilizzato nel capitolo successivo per l'estrazione delle caratteristiche nel dominio del tempo e della frequenza.

3 Estrazione feature dalla timetable

Una volta generata la struttura dati nella tabella timetable (`dataTable`), in cui a ogni segnale è associata la relativa evoluzione temporale, il passo successivo è consistito nell'estrazione delle caratteristiche (feature) rilevanti. Queste metriche costituiscono la base informativa su cui il classificatore dovrà operare.

3.1 Impostazione della Frame Policy

Per analizzare correttamente i segnali, è stata definita una specifica strategia di segmentazione temporale, nota come *Frame Policy*. L'approccio di estrarre feature sull'intera durata del volo avrebbe fornito una rappresentazione media troppo generica, nascondendo le variazioni dinamiche e i transitori locali. Al contrario, l'estrazione su porzioni specifiche (*frame*) permette di catturare l'evoluzione temporale del guasto e di moltiplicare il numero di campioni disponibili per l'addestramento.

Per questo progetto, è stata impostata una *lunghezza del frame pari a 1 secondo*. Considerando la frequenza di campionamento di sincronizzazione $F_s = 350$ Hz, ogni frame contiene esattamente 350 campioni. Questa dimensione è stata scelta come compromesso ottimale:

- È sufficientemente lunga da garantire una buona risoluzione spettrale per l'analisi in frequenza (permettendo di distinguere i picchi armonici).
- È sufficientemente breve da approssimare il segnale come stazionario all'interno della finestra.

Di conseguenza, i log di volo sono stati processati catturando frame consecutivi con la periodicità di 1 secondo, massimizzando l'estrazione di informazioni utili senza introdurre ridondanza eccessiva o valori non validi (NaN).

3.2 Processo di Estrazione delle Feature

Il processo di estrazione è stato inizializzato utilizzando lo strumento interattivo *Diagnostic Feature Designer* di MATLAB. Questo tool ha permesso di esplorare preliminarmente i segnali nel dominio del tempo e della frequenza e di configurare gli algoritmi di calcolo (come il metodo di Welch e la peak detection).

Una volta validata la configurazione, è stata utilizzata la funzionalità di **Code Generation** del Designer per esportare la logica in una funzione MATLAB autonoma, denominata `diagnosticFeaturesHighFreq2Peak`.

Le feature estratte sono state successivamente sottoposte a un processo di selezione (Ranking) utilizzando separatamente le metriche **One-Way ANOVA** e

Kruskal-Wallis, al fine di individuare le variabili con il maggior potere discriminante tra le classi di guasto. La corretta estrazione e selezione delle feature costituisce il nucleo fondamentale del sistema diagnostico, infatti nessun classificatore, per quanto complesso, può fornire risultati accurati se alimentato con dati che non evidenziano chiaramente l'anomalia.

Di seguito vengono descritti nel dettaglio i passaggi per l'estrazione delle feature nei domini del tempo e della frequenza.

3.3 Feature nel dominio del tempo

Le feature nel dominio del tempo sono metriche statistiche calcolate direttamente sulla forma d'onda del segnale. Sono state scelte tutte le feature disponibili ad eccezione di quelle relative alle componenti armoniche.

Per ogni segnale sincronizzato sono state estratte le seguenti caratteristiche statistiche :

- **Mean**: valore medio del segnale.
- **Std (Deviazione Standard)**: misura la dispersione del segnale attorno alla media.
- **RMS (Root Mean Square)**: errore quadratico medio del segnale.
- **Peak Value**: Il valore assoluto massimo raggiunto dal segnale nella finestra temporale.
- **Crest Factor**: Rapporto tra il valore di picco e l'RMS. Indica quanto il segnale è impulsivo.
- **Shape Factor**: Rapporto tra l'RMS e la media dei valori assoluti.
- **Impulse Factor**: Rapporto tra il valore di picco e la media assoluta.
- **Kurtosis**: lunghezza delle code di una distribuzione del segnale o, equivalentemente, quanto è incline il segnale ai valori anomali
- **Skewness**: misura l'asimmetria della distribuzione rispetto alla media.
- **Clearance Factor**: rapporto tra il picco e il quadrato della media delle radici quadrate delle ampiezze. È una metrica sensibile specifica per macchine rotanti.

3.4 Feature nel dominio della frequenza

L'analisi nel dominio della frequenza è stata condotta esclusivamente sulle **variabili acquisite ad alta frequenza**, ovvero i segnali accelerometrici (`acc_x`, `acc_y`, `acc_z`) e giroscopici (`gyr_x`, `gyr_y`, `gyr_z`) provenienti dalla IMU. Le altre grandezze del dataset, avendo dinamiche più lente, non presentano un contenuto spettrale significativo per la diagnosi vibrazionale.

All'interno del *Diagnostic Feature Designer*, la trasformazione è stata effettuata mediante la stima della **Densità Spettrale di Potenza (PSD)** utilizzando il metodo di Welch.

A titolo esemplificativo, nelle Figure 1 e 2 sono riportati gli spettri di potenza calcolati per una variabile accelerometrica (`acc_z`) e una giroscopica (`gyr_x`). I grafici mostrano l'andamento dell'energia del segnale al variare della frequenza (rappresentata su scala logaritmica sull'asse delle ascisse). I marcatori pieni presenti sulle curve evidenziano i picchi rilevati automaticamente dallo strumento. Come si evince dalle figure, le componenti spettrali più significative e i picchi di risonanza si concentrano chiaramente in specifiche bande di frequenza (tipicamente tra 10 Hz e 100 Hz), giustificando l'analisi mirata in questo dominio e la scelta della banda operativa.

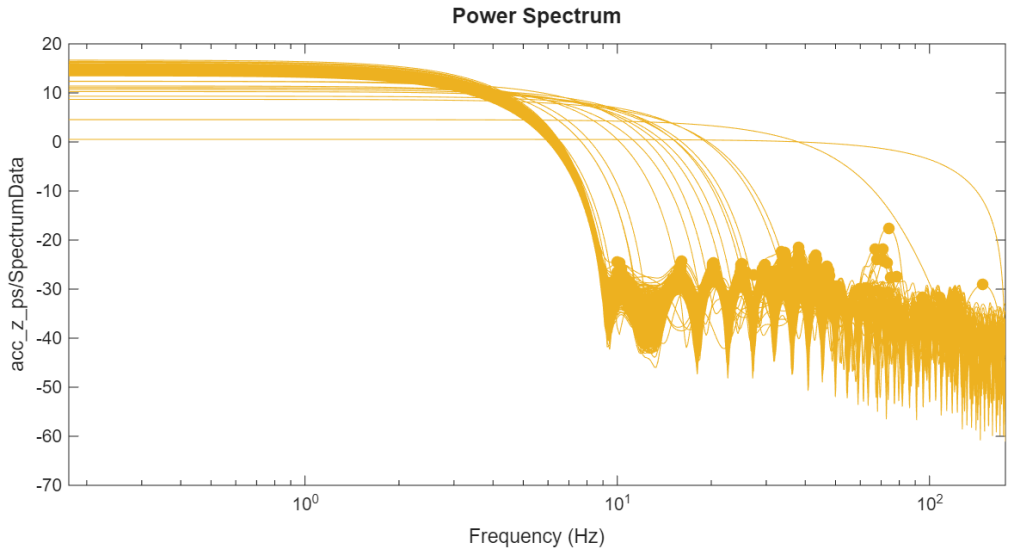


Figura 1: Spettro di potenza (stimato con il metodo di Welch) per il segnale accelerometrico lungo l'asse Z (`acc_z`)

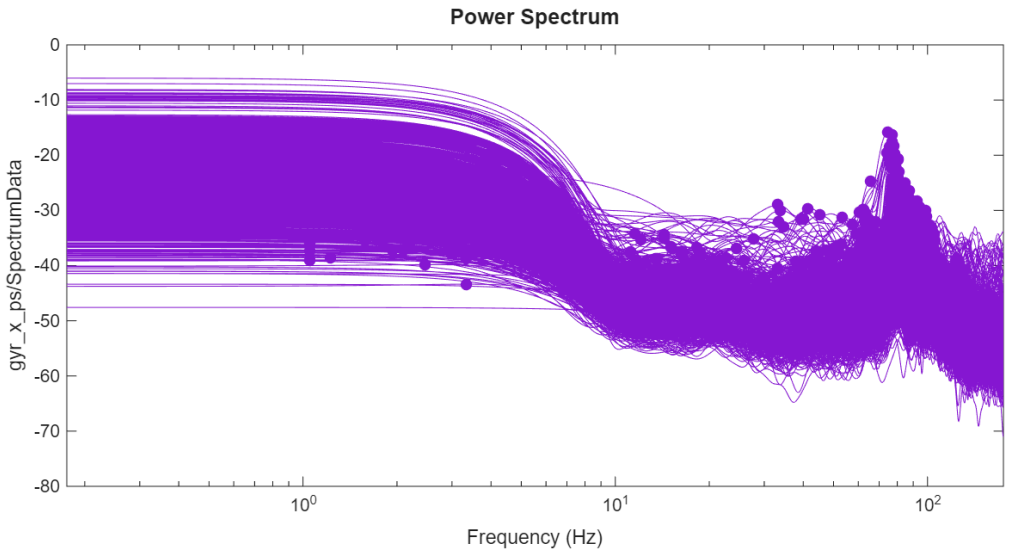


Figura 2: Spettro di potenza (stimato con il metodo di Welch) per il segnale giroscopico attorno all'asse X (`gyr_x`)

3.4.1 Configurazione dell'estrazione dei Picchi (Spectral Peaks)

Per individuare le caratteristiche distintive del danno alla pala, è stata modificata la configurazione standard di estrazione delle feature spettrali direttamente nell'interfaccia del Diagnostic Feature Designer. Invece di limitare l'analisi al solo picco dominante (impostazione di default), è stato selezionato il parametro per estrarre i **primi due picchi** più energetici dello spettro.

Di conseguenza, per ciascuno dei 6 segnali IMU, sono state estratte le seguenti feature:

- **PeakAmp1, PeakAmp2**: Ampiezza (energia della vibrazione) del primo e del secondo picco.
- **PeakFreq1, PeakFreq2**: Frequenza del primo e del secondo picco.
- **BandPower**: Potenza totale del segnale vibrazionale.

3.5 Migliori Feature

L'estrazione automatica ha prodotto un numero elevato di feature (diverse centinaia), molte delle quali ridondanti o poco informative. Per selezionare solo le variabili più discriminanti, sono stati utilizzati due test statistici distinti per valutare la capacità di ogni feature di separare le classi di guasto.

3.5.1 Ranking tramite One-Way ANOVA

L'Analisi della Varianza (ANOVA) è stata utilizzata come metodo parametrico per valutare la significatività statistica delle feature estratte. Questo test calcola il rapporto tra la varianza *tra* le classi e la varianza *all'interno* delle classi , producendo un valore noto come **F-score**.

Un F-score elevato indica che la feature varia significativamente al variare della classe di guasto (Sano, 5%, 10%), mantenendosi invece stabile all'interno della stessa classe.

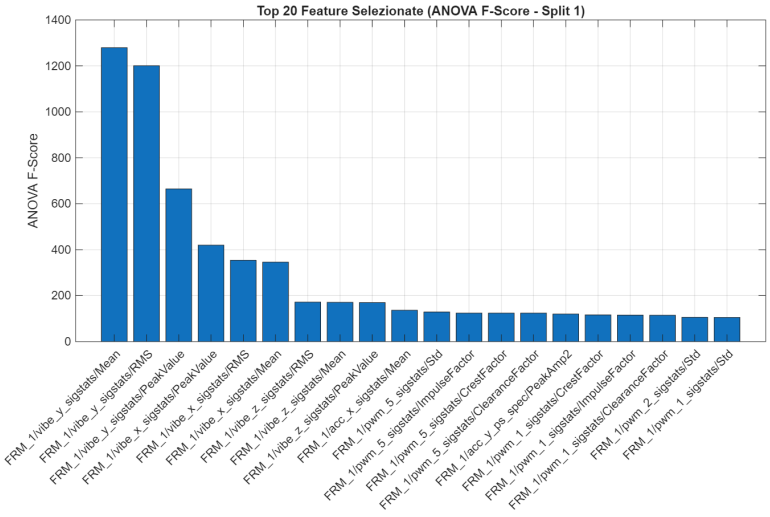


Figura 3: Ranking delle migliori 20 feature selezionate da One-Way ANOVA

Come si può osservare nel grafico in Figura 3, le barre rappresentano l'importanza relativa di ciascuna caratteristica. Le feature con i punteggi più alti (in alto nel grafico) sono quelle che offrono la migliore separabilità tra le classi. Sulla base di questi punteggi, sono state selezionate le **Top 20 feature** da includere nel dataset di addestramento.

3.5.2 Ranking tramite Kruskal-Wallis

Il test di Kruskal-Wallis è l'equivalente non parametrico dell'ANOVA. Invece di confrontare le medie, confronta le mediane dei gruppi. Questo test è più robusto nel caso in cui le feature non seguano una distribuzione normale (gaussiana), situazione comune nei dati reali affetti da rumore. Anche per questo metodo è stato generato un ranking indipendente e sono state selezionate le **Top 20 feature**.

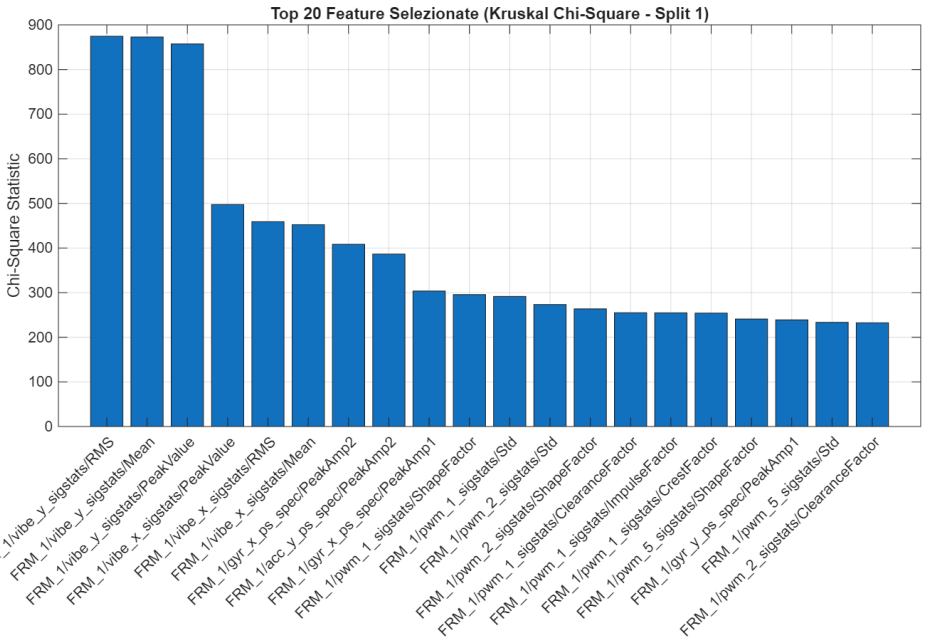


Figura 4: Ranking delle migliori 20 feature selezionate da Kruskal-Wallis

3.5.3 Definizione dei Set di Feature per il Confronto

Anziché unire i risultati delle due tecniche di ranking, in questo lavoro si è scelto di mantenere separati i gruppi di feature selezionate per effettuare un confronto diretto tra le due metodologie.

Sono stati pertanto definiti due set di feature distinti, che verranno utilizzati parallelamente per addestrare i classificatori:

- **Set ANOVA**: Costituito esclusivamente dalle **Top 20 feature** selezionate tramite il test parametrico One-Way ANOVA (basato sul confronto delle medie).
- **Set Kruskal-Wallis**: Costituito esclusivamente dalle **Top 20 feature** selezionate tramite il test non parametrico (basato sul confronto delle mediane).

Questa strategia ha l'obiettivo di determinare quale dei due test statistici sia più efficace nel selezionare le caratteristiche discriminanti per i guasti alle pale. In particolare, si vuole verificare se l'approccio non parametrico (Kruskal-Wallis) offra vantaggi rispetto all'ANOVA, dato che i segnali di guasto reali potrebbero non rispettare sempre le assunzioni di normalità (distribuzione gaussiana) richieste dai test parametrici.

4 Classificazione dei guasti

Una volta completata l'estrazione e la selezione delle caratteristiche ottimali, si è passati alla fase di modellazione predittiva. L'obiettivo di questo capitolo è addestrare diversi classificatori affinché imparino a mappare le feature estratte nello stato di salute effettivo del drone (Classe 0: Sano, Classe 5: Guasto 5%, Classe 10: Guasto 10%).

4.1 Divisione del Dataset

La valutazione dell'efficacia dei classificatori richiede che i modelli vengano testati su dati mai visti durante la fase di addestramento. Per ottenere una stima affidabile della capacità di generalizzazione, è stata implementata una procedura di **Validazione Incrociata Ripetuta** (Repeated Holdout).

Utilizzando la funzione MATLAB `cvpartition`, il dataset delle feature è stato partizionato applicando le seguenti regole:

- **Proporzione 90/10**: In ogni iterazione, il 90% dei dati è stato assegnato al Training Set (per l'addestramento e la validazione interna), mentre il restante 10% è stato isolato per formare il Test Set (per la validazione finale).
- **Stratificazione (`'Stratify', true`)**: La divisione è avvenuta mantenendo inalterate le proporzioni originali delle classi. Questo garantisce che sia il Training Set sia il Test Set contengano un numero bilanciato di campioni sani e guasti, evitando distorsioni (bias) nell'apprendimento.
- **Iterazioni Multiple (`nSplits = 10`)**: L'intero processo di partizionamento è stato ripetuto generando 10 split casuali e indipendenti. Ogni modello di classificazione è stato quindi addestrato e testato 10 volte, una per ogni split.

4.2 Metriche di Valutazione

Al fine di confrontare in modo oggettivo i diversi algoritmi e i diversi set di feature (ANOVA vs Kruskal-Wallis), sono state calcolate le seguenti metriche di prestazione basate sui risultati dei 10 split:

1. **Accuratezza Media e Deviazione Standard**: Per ogni classificatore, è stata calcolata l'accuratezza (percentuale di predizioni corrette) su ciascuno dei 10 Test Set. I risultati finali sono riportati sotto forma di media statistica accompagnata dalla sua deviazione standard ($\mu \pm \sigma$). Una deviazione standard bassa indica che il modello è stabile e non dipende troppo dalla specifica partizione dei dati.

2. **Matrice di Confusione Cumulativa (Aggregata)**: Per valutare gli errori specifici, i risultati delle predizioni ottenuti sui 10 Test Set indipendenti sono stati concatenati tra loro. Questo ha permesso di generare un'unica Matrice di Confusione Globale che rappresenta il comportamento medio del modello sull'intero dataset.

4.3 Classificatori Utilizzati

Per esplorare diverse logiche di apprendimento, sono state addestrate e confrontate le seguenti tipologie di algoritmi:

- **Alberi Decisionali (Decision Tree)**: Modelli interpretabili ("white box") che basano la predizione su una serie di regole gerarchiche del tipo "if-then-else".
- **K-Nearest Neighbors (KNN)**: Modello basato sulla distanza spaziale tra i campioni nel dominio delle feature.
- **Support Vector Machine (SVM)**: Algoritmo robusto che cerca l'iperpiano di separazione a massimo margine tra le classi, utilizzando kernel specifici per gestire non-linearità.
- **Random Forest (Ensemble)**: Aggregazione di molteplici alberi decisionali per migliorare la stabilità predittiva e ridurre l'overfitting.
- **Reti Neurali (Neural Networks)**: Modelli basati su architetture a strati capaci di apprendere rappresentazioni profonde e relazioni altamente non lineari tra le feature e lo stato di salute del drone.

4.4 Alberi decisionali

L'Albero Decisionale (Decision Tree) è un algoritmo di apprendimento supervisionato non parametrico che modella le decisioni attraverso una struttura gerarchica. Il modello partiziona iterativamente lo spazio delle feature creando dei nodi decisionali basati su semplici regole di soglia .

Il principale vantaggio di questo classificatore risiede nella sua natura white-box: a differenza di modelli più complessi e opachi, il percorso logico che porta a una specifica predizione è completamente trasparente e facilmente interpretabile da un operatore umano.

Configurazione del Modello

Per l'addestramento in ambiente MATLAB è stata utilizzata la funzione `fitctree`. Al fine di ottimizzare le prestazioni e ridurre la distorsione del modello, le configurazioni principali adottate sono state le seguenti:

- **Criterio di splitting (`'PredictorSelection', 'curvature'`)**: È stato utilizzato il test di curvatura per selezionare la feature ottimale per ogni divisione. Questo approccio mitiga il bias tipico degli alberi decisionali standard, che tendono a favorire le variabili con molti livelli o valori unici.

- **Surrogate splits disabilitati (`'Surrogate', 'off'`)**: Poiché il dataset è stato precedentemente pulito dai valori mancanti (NaN) nella fase di preprocessing, non è stato necessario calcolare regole decisionali surrogate, riducendo così la complessità computazionale e i tempi di addestramento.

4.4.1 Due classi

Il primo scenario analizzato è la classificazione binaria, il cui obiettivo è la semplice rilevazione dell'anomalia (*Fault Detection*). Per questo scenario, i dati sono stati rietichettati unendo le classi di guasto originali (5% e 10%) in un'unica macroclasse indicante la condizione generale di "Guasto" (Classe 1), contrapposta alla condizione nominale "Sano" (Classe 0).

Di seguito è riportata la matrice di confusione cumulativa ottenuta sui 10 split per il modello addestrato utilizzando le feature selezionate tramite il test statistico ANOVA (Figura 5).

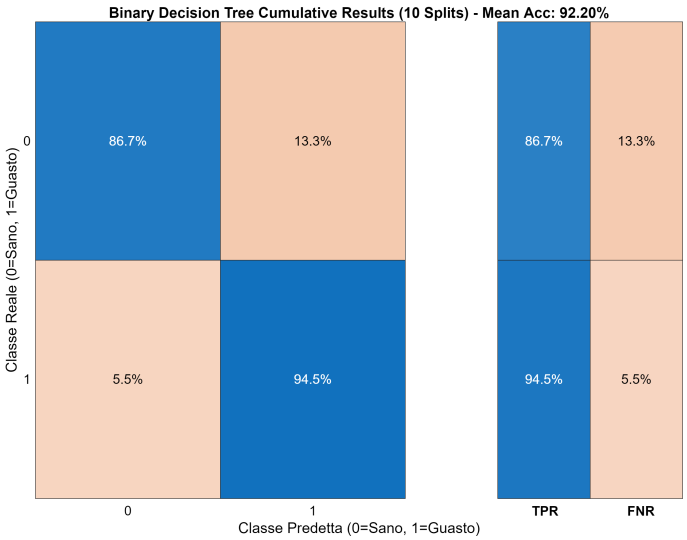


Figura 5: Confusion Matrix del modello Decision Tree per due classi (Set ANOVA).

La figura 6 illustra invece i risultati della classificazione binaria quando il modello viene addestrato con il set di feature derivato dal test non parametrico di Kruskal-Wallis.

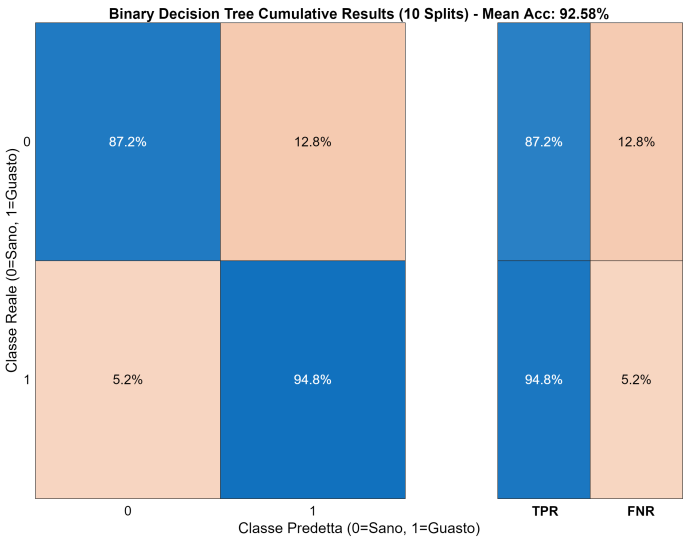


Figura 6: Confusion Matrix del modello Decision Tree per due classi (Set Kruskal-Wallis).

4.4.2 Tre classi

Il secondo scenario affronta il problema della diagnosi a tre classi, in cui il classificatore deve discriminare tra tre stati operativi distinti: Sano (0), Guasto al 5% (5) e Guasto al 10% (10).

La prima matrice di confusione per questo scenario multiclasse rappresenta le prestazioni del modello basato sulle feature selezionate tramite ANOVA (Figura 7).

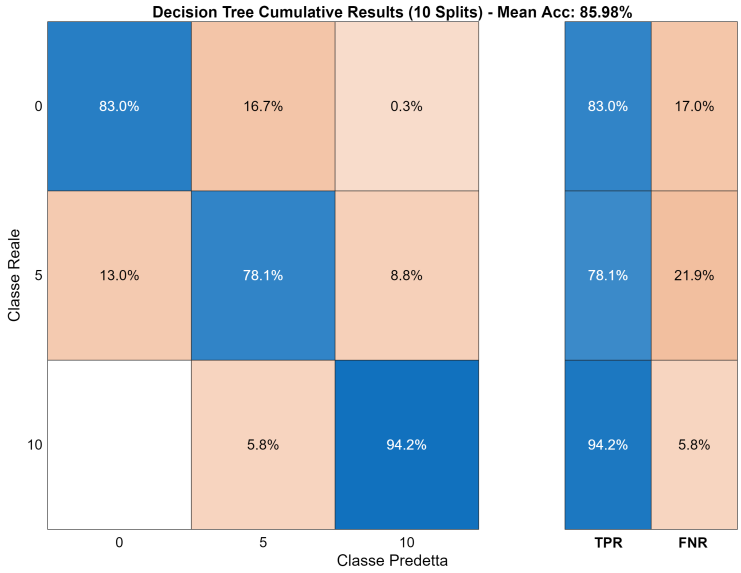


Figura 7: Confusion Matrix del modello Decision Tree per tre classi (Set ANOVA).

Infine, viene presentata la matrice di confusione multiclasse relativa al modello addestrato impiegando il sottoinsieme di feature individuate tramite Kruskal-Wallis (Figura 8).

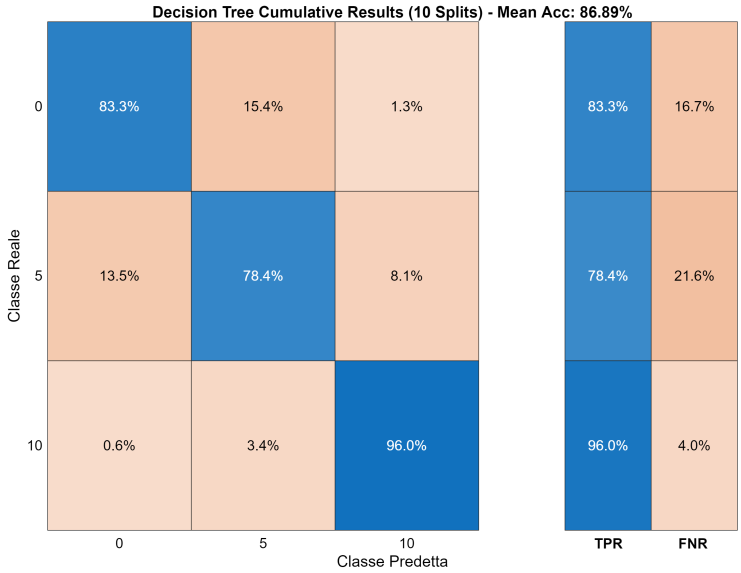


Figura 8: Confusion Matrix del modello Decision Tree per tre classi (Set Kruskal-Wallis).

4.5 K-Nearest Neighbors (KNN)

L'algoritmo K-Nearest Neighbors (KNN) è un metodo di classificazione non parametrico basato sull'apprendimento lazy. A differenza di altri modelli, il KNN non costruisce una funzione analitica durante la fase di addestramento, ma memorizza l'intero dataset. Quando è necessario classificare un nuovo campione , l'algoritmo calcola la distanza tra questo e tutti i campioni presenti nel training set, assegnando infine al nuovo dato la classe maggioritaria tra i suoi K vicini più prossimi.

Configurazione del Modello

Per l'implementazione del KNN in MATLAB, è stata utilizzata la funzione `fitcknn`. Le impostazioni fondamentali per ottimizzare l'algoritmo sono state:

- **Numero di vicini (`'NumNeighbors', 10`)**: Si è scelto empiricamente un valore di $K = 10$ per garantire un buon bilanciamento tra la sensibilità al rumore locale (tipica di K troppo piccoli) e la perdita di risoluzione decisionale (tipica di K troppo grandi).

- **Metrica di distanza** (`'Distance', 'euclidean'`): È stata adottata la distanza euclidea classica per misurare la similarità spaziale nel dominio delle feature.
- **Standardizzazione** (`'Standardize', 1`): Poiché la distanza euclidea è fortemente sensibile alla scala numerica delle variabili, è stato cruciale abilitare la standardizzazione automatica delle feature (media nulla e varianza unitaria). Questo garantisce che variabili con ampiezze molto diverse abbiano tutte lo stesso peso nel calcolo della distanza.

4.5.1 Due classi

Anche per il modello KNN, il primo scenario affrontato è stato quello binario (*Sano* vs *Guasto*). I campioni relativi ai guasti del 5% e del 10% sono stati uniti in una singola classe per l'addestramento e il testing.

Di seguito viene presentata la matrice di confusione cumulativa (Figura 9) ottenuta aggregando i risultati dei 10 split. Il modello analizzato in questa prima figura è stato addestrato esclusivamente sul set di feature derivato dal ranking ANOVA.

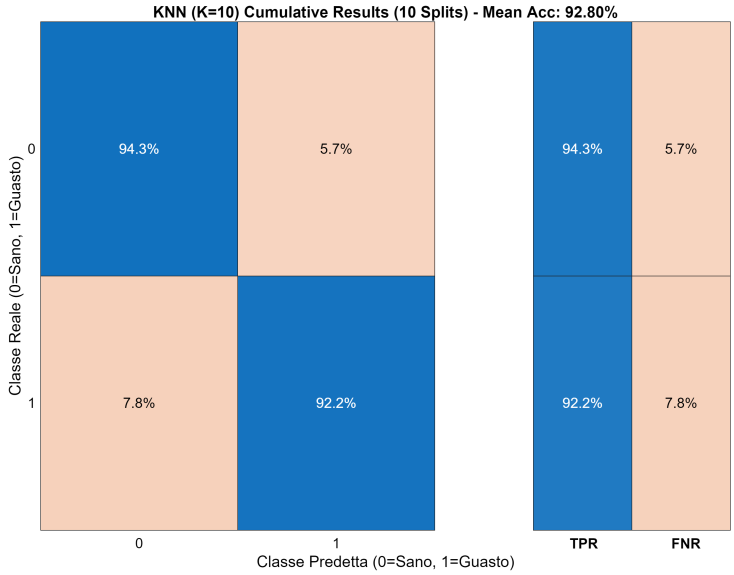


Figura 9: Confusion Matrix del modello KNN per due classi (Set ANOVA).

Per confronto, la figura 10 mostra le prestazioni del modello KNN binario quando alimentato con le feature selezionate tramite il test non parametrico di Kruskal-Wallis.

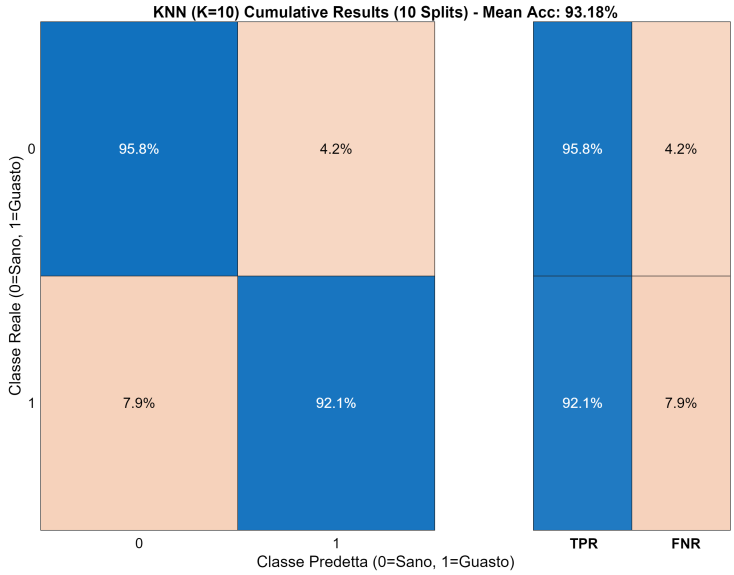


Figura 10: Confusion Matrix del modello KNN per due classi (Set Kruskal-Wallis).

4.5.2 Tre classi

Successivamente, il modello KNN è stato testato sulla problematica multi-classe, con l'obiettivo di separare correttamente i voli nominali, i guasti incipienti (5%) e i guasti severi (10%). La capacità del KNN di risolvere questo task dipende fortemente dalla reale separabilità geometrica dei tre gruppi all'interno dello spazio n-dimensionale delle feature selezionate.

La figura 11 espone la matrice di confusione cumulativa per lo scenario a tre classi, utilizzando il set di feature ANOVA.

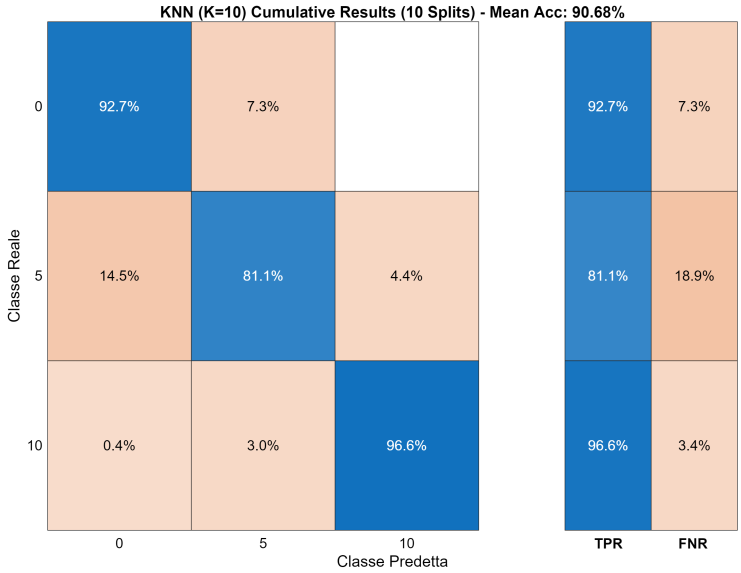


Figura 11: Confusion Matrix del modello KNN per tre classi (Set ANOVA).

Infine, viene riportato il comportamento predittivo del KNN addestrato per discriminare le tre classi di guasto sfruttando il sottoinsieme di variabili del test di Kruskal-Wallis (Figura 12).

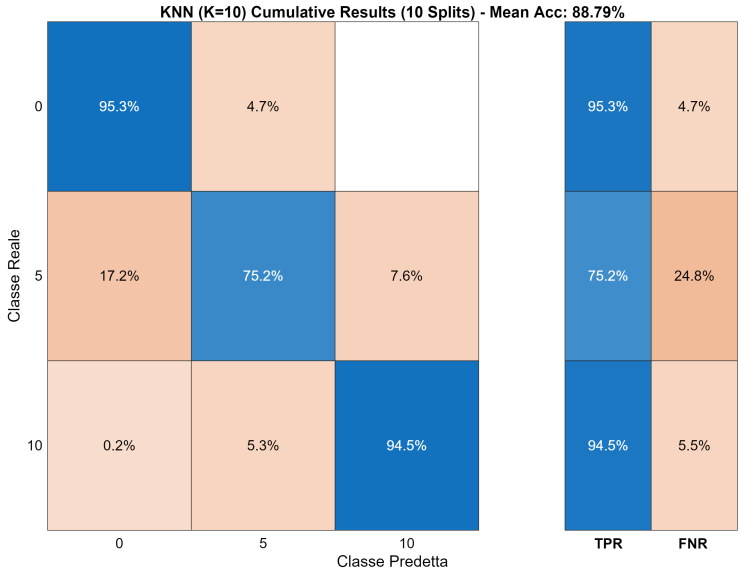


Figura 12: Confusion Matrix del modello KNN per tre classi (Set Kruskal-Wallis).

4.6 Support Vector Machine (SVM)

Le Support Vector Machine (SVM) sono una classe di algoritmi di apprendimento supervisionato ampiamente utilizzati per problemi di classificazione complessi. L'obiettivo primario di una SVM è identificare l'iperpiano decisionale ottimale che separi le diverse classi massimizzando il margine di distanza tra l'iperpiano stesso e i campioni di addestramento più vicini (noti come *vettori di supporto*).

Quando i dati non sono linearmente separabili nello spazio originale delle feature, l'algoritmo SVM utilizza il kernel trick: mappa i dati in uno spazio a dimensionalità superiore in cui è possibile tracciare un iperpiano separatore lineare, permettendo così di risolvere problemi con confini decisionali complessi e non lineari.

Configurazione del Modello

Per addestrare i modelli SVM, l'ambiente MATLAB offre due funzioni principali in base al numero di classi: `fitcsvm` per problemi binari e `fitcecoc` per problemi multi-classe (utilizzando un approccio *Error-Correcting Output Codes*, tipicamente One-vs-One). In entrambi gli scenari, la configurazione dell'algoritmo ha mantenuto le seguenti impostazioni fondamentali:

- **Funzione Kernel (`'KernelFunction', 'polynomial'`)**: Si è optato per un kernel polinomiale anziché uno lineare o RBF (Radial Basis Function).
- **Ordine del Polinomio (`'PolynomialOrder', 2`)**: È stato selezionato un polinomio di secondo grado (SVM Quadratica). Questa scelta offre un buon compromesso tra la capacità di modellare relazioni non lineari e la prevenzione dell'overfitting che potrebbe derivare da gradi polinomiali più elevati.
- **Standardizzazione (`'Standardize', true`)**: Poiché la SVM cerca di massimizzare un margine geometrico, è essenziale che tutte le feature siano scalate per evitare che quelle con range numerici più ampi dominino il calcolo della distanza. L'attivazione di questo parametro normalizza automaticamente i dati (media zero e varianza unitaria) prima dell'addestramento.

4.6.1 Due classi

Nello scenario di *Fault Detection* binaria, i voli con danno al 5% e al 10% sono stati combinati nella classe unitaria "Guasto" (1), per distinguerli dalle condizioni "Sano" (0). L'addestramento è stato effettuato mediante la funzione `fitcsvm`.

La figura 13 riporta la matrice di confusione cumulativa (ottenuta dalla somma dei 10 split di test) per la SVM Quadratica addestrata sul set di feature individuato dall'ANOVA.

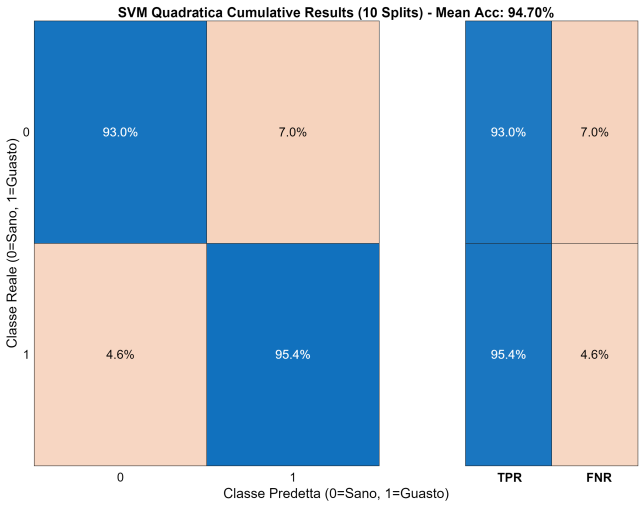


Figura 13: Confusion Matrix per SVM Quadratica due classi (Set ANOVA).

Per consentire il confronto, viene presentata di seguito la matrice di confusione derivante dall'utilizzo delle feature estratte tramite il test di Kruskal-Wallis (Figura 14).

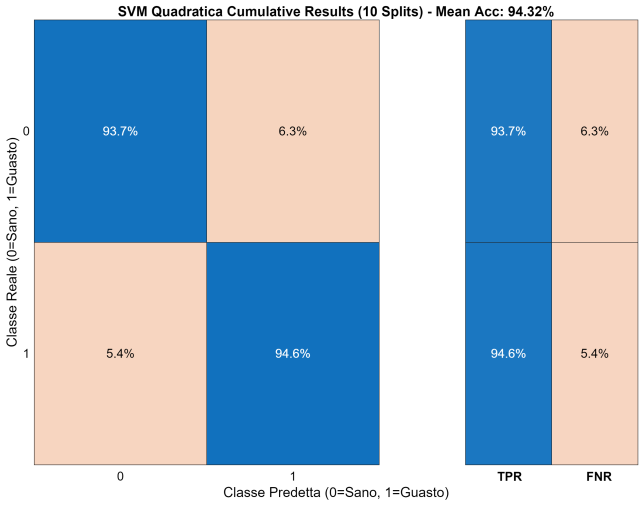


Figura 14: Confusion Matrix per SVM Quadratica due classi (Set Kruskal-Wallis).

4.6.2 Tre classi

Per la classificazione multi-classe, è stato impiegato un modello ECOC (`fitcecoc`) basato su *learner* SVM quadratici, con lo scopo di separare simultaneamente i voli nominali (0), i guasti lievi (5) e i guasti severi (10).

La prima matrice di confusione (Figura 15) per questo task a tre classi illustra i risultati ottenuti impiegando il set di feature parametrico (ANOVA).

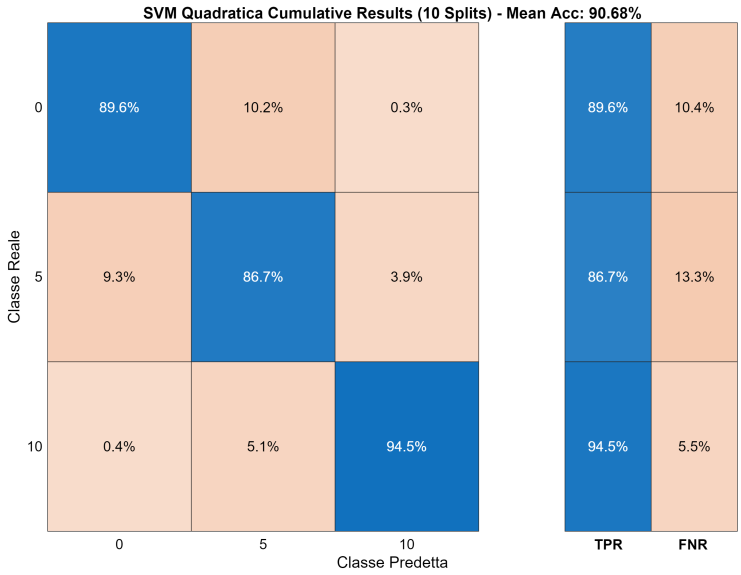


Figura 15: Confusion Matrix del modello SVM Quadratica per tre classi (Set ANOVA).

Infine, la figura 16 mostra le prestazioni predittive della SVM Quadratica multi-classe quando viene addestrata con il set di feature non parametrico (Kruskal-Wallis).

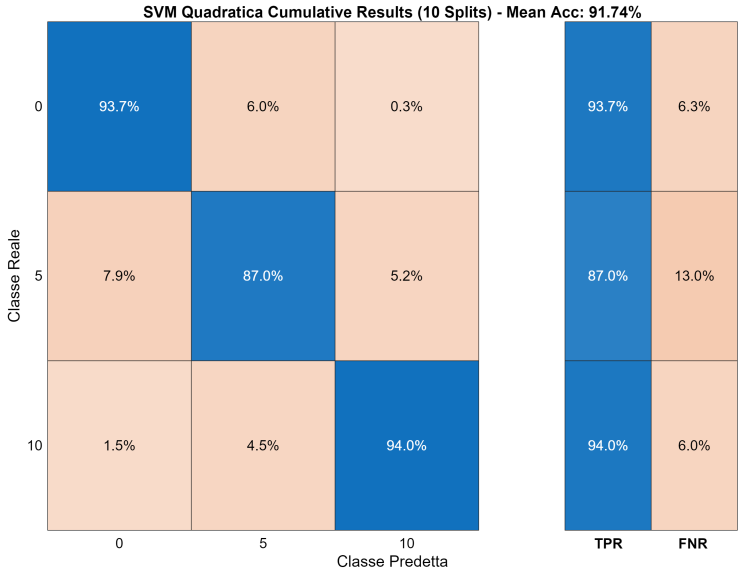


Figura 16: Confusion Matrix del modello SVM Quadratica per tre classi (Set Kruskal-Wallis).

4.7 Random Forest

Il Random Forest è un algoritmo di apprendimento supervisionato basato sulla tecnica di *Ensemble Learning*, progettato per superare i limiti intrinseci dei singoli alberi decisionali, come la tendenza all'overfitting e l'alta sensibilità alle variazioni nel set di addestramento.

Questo modello costruisce una foresta di molteplici alberi decisionali indipendenti durante la fase di addestramento. La predizione finale (Sano, Guasto 5% o Guasto 10%) viene stabilita tramite un meccanismo di votazione a maggioranza (*majority voting*) tra tutti gli alberi che compongono l'ensemble. Questo approccio collettivo garantisce previsioni molto più robuste, stabili e generalizzabili rispetto al singolo albero decisionale.

Configurazione del Modello

L'implementazione in MATLAB è stata realizzata tramite la funzione `fitcensemble`. Le principali scelte progettuali e parametriche derivate dal codice sono:

- **Metodo di aggregazione (`'Method', 'Bag'`)**: È stata utilizzata la tecnica di *Bagging* (Bootstrap Aggregating). Ogni albero della foresta viene addestrato su un sottoinsieme casuale (estratto con reinserimento) del dataset di training originale.
- **Numero di Alberi (`'NumLearningCycles'`)**: Il numero di *learners* è stato ottimizzato in base alla complessità del problema, impostando 50 alberi per lo scenario binario e 30 alberi per quello multi-classe, garantendo un buon equilibrio tra tempi di calcolo e accuratezza.

4.7.1 Due classi

Nella configurazione di *Fault Detection* a due classi, le condizioni di guasto (5% e 10%) sono state accorpate per istruire il modello a riconoscere la semplice anomalia contro il funzionamento sano.

Di seguito è riportata la matrice di confusione cumulativa per il Random Forest binario addestrato sulle feature classificate tramite il test ANOVA (Figura 17).

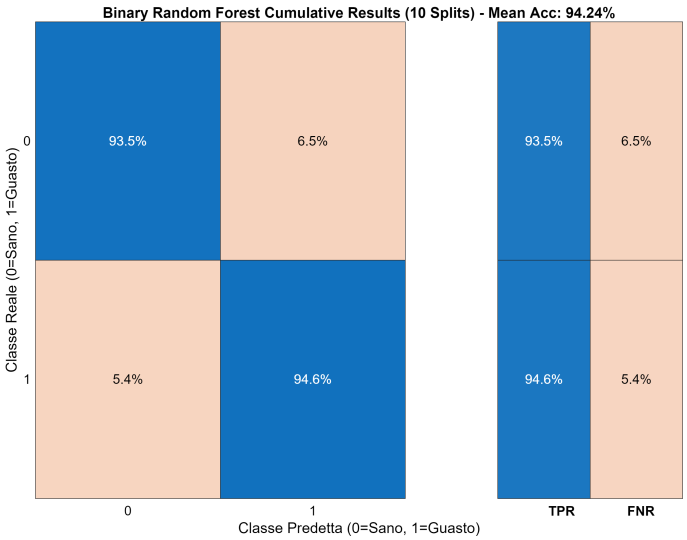


Figura 17: Confusion Matrix del modello Random Forest per due classi (Set ANOVA).

Per valutare l'impatto del set di dati di input, la figura 18 mostra le prestazioni della medesima architettura Random Forest, ma addestrata utilizzando le migliori feature individuate dal test di Kruskal-Wallis.

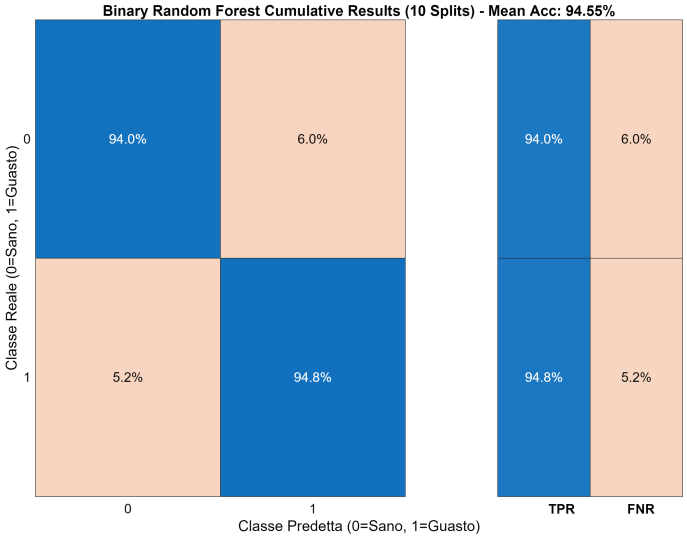


Figura 18: Confusion Matrix del modello Random Forest per due classi (Set Kruskal-Wallis).

4.7.2 Tre classi

Il problema multi-classe richiede alla Random Forest di identificare non solo la presenza del danno, ma anche la sua estensione, separando le classi Sano, Guasto 5% e Guasto 10%.

La figura 19 presenta la matrice di confusione aggregata risultante dall'addestramento multi-classe basato sul sottoinsieme di feature parametrico (ANOVA).

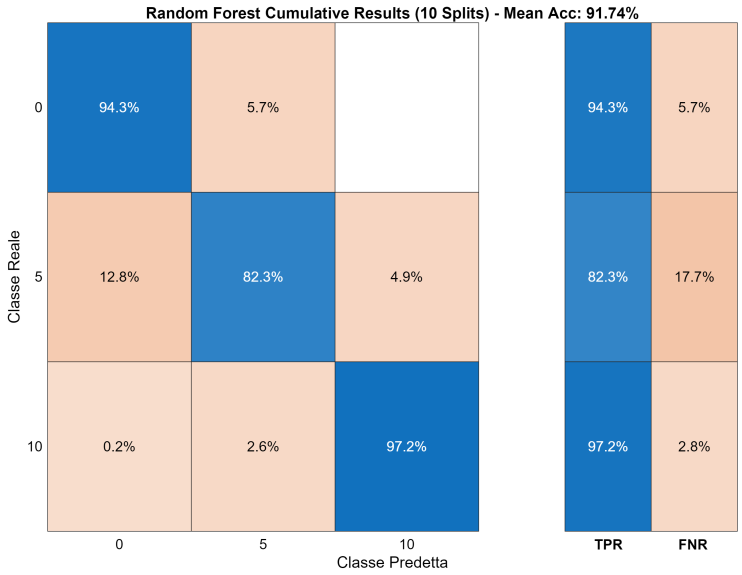


Figura 19: Confusion Matrix del modello Random Forest per tre classi (Set ANOVA).

Infine, viene proposta la matrice di confusione multi-classe (Figura 20) per il modello Random Forest che ha impiegato le feature di addestramento estratte mediante il test non parametrico (Kruskal-Wallis).

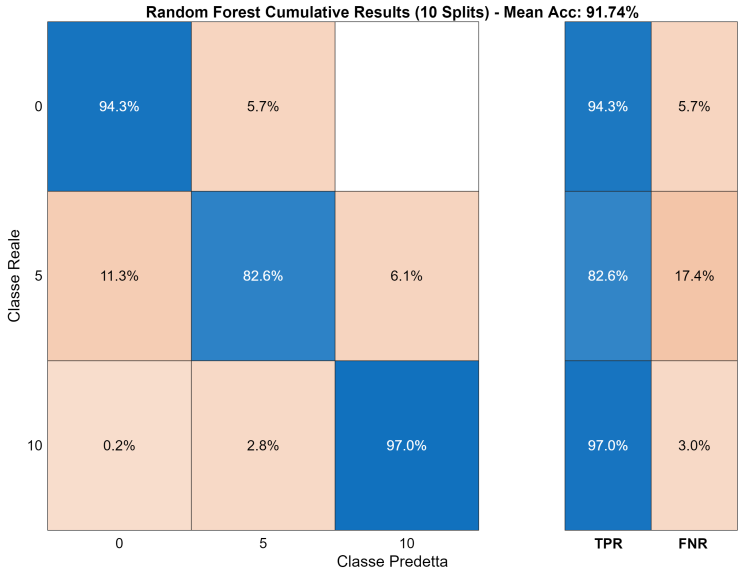


Figura 20: Confusion Matrix del modello Random Forest per tre classi (Set Kruskal-Wallis).

4.8 Reti Neurali

Le Neural Networks sono modelli computazionali ispirati alle reti neurali biologiche, in grado di apprendere rappresentazioni profonde e identificare pattern altamente complessi e non lineari all'interno dei dati. Nel contesto del Machine Learning tradizionale (non Deep Learning), ci si riferisce tipicamente ad architetture *Feedforward Multi-Layer Perceptron (MLP)*, costituite da un livello di input, uno o più livelli nascosti (*hidden layers*) e un livello di output.

A differenza di modelli come il Decision Tree o il KNN, le reti neurali operano come delle black-box: il processo decisionale è distribuito sui pesi sinaptici dell'intera rete, rendendo complesso risalire alla motivazione esatta dietro una singola predizione, ma offrendo al contempo capacità predittive estremamente potenti.

Configurazione del Modello

Per l'addestramento della rete neurale in MATLAB è stata impiegata la funzione `fitcnet`, configurando un'architettura Feedforward pienamente connessa con le seguenti caratteristiche:

- **Dimensione dei Livelli Nascosti (`'LayerSizes', 25`)**: È stato impostato un singolo livello nascosto composto da 25 neuroni. Questa configurazione è abbastanza contenuta da prevenire il rischio di overfitting sui dati di addestramento.
- **Funzione di Attivazione (`'Activations', 'relu'`)**: Per introdurre nonlinearità nella rete e mitigare il problema della scomparsa del gradiente (*vanishing gradient*) durante l'addestramento, è stata scelta la funzione ReLU (Rectified Linear Unit) per i neuroni del livello nascosto.
- **Standardizzazione (`'Standardize', true`)**: Similmente a quanto visto per KNN e SVM, la normalizzazione dei dati in ingresso è un passaggio **fondamentale** per l'addestramento delle reti neurali. Permette agli algoritmi di ottimizzazione basati sul gradiente di convergere più rapidamente e impedisce che le feature con scale numeriche più grandi saturino l'attivazione dei neuroni.

4.8.1 Due classi

Anche per le reti neurali, il primo livello di analisi si è concentrato sulla rilevazione di anomalia pura (Sano vs Guasto generico), unendo le classi corrispondenti ai danni del 5% e del 10%.

La figura 21 riporta la matrice di confusione cumulativa calcolata sui 10 test set indipendenti per la Rete Neurale addestrata sulle migliori variabili estratte tramite il test ANOVA.

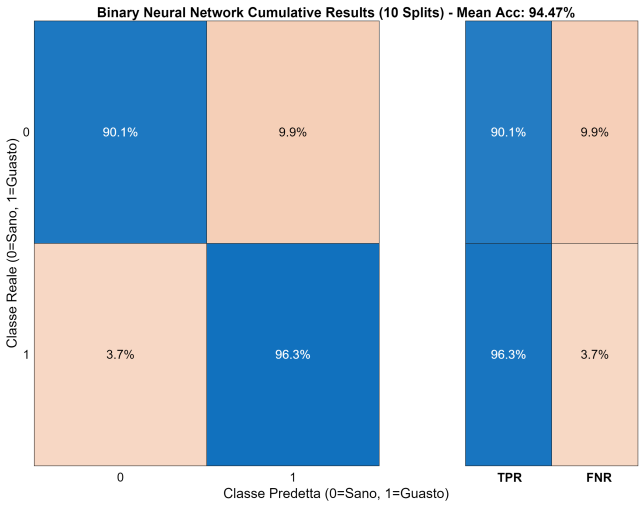


Figura 21: Confusion Matrix per la Neural Network due classi (Set ANOVA).

Di seguito viene mostratala matrice di confusione relativa allo stesso task binario, ma eseguito fornendo alla rete le feature selezionate tramite il test di Kruskal-Wallis (Figura 22).

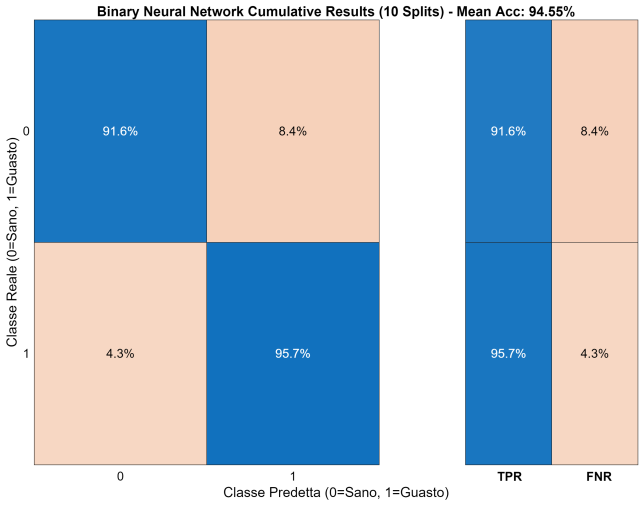


Figura 22: Confusion Matrix per la Neural Network due classi (Set Kruskal-Wallis).

4.8.2 Tre classi

Infine, la Neural Network è stata messa alla prova sul problema di classificazione a multi-classe, con l'obiettivo di mappare le feature vibrazionali in tre specifici stati di salute della pala: Sano, Guasto 5% e Guasto 10%.

La prima delle due matrici sottostanti (Figura 23) sintetizza le capacità predittive multi-classe della rete neurale alimentata con il set di feature di derivazione parametrica (ANOVA).

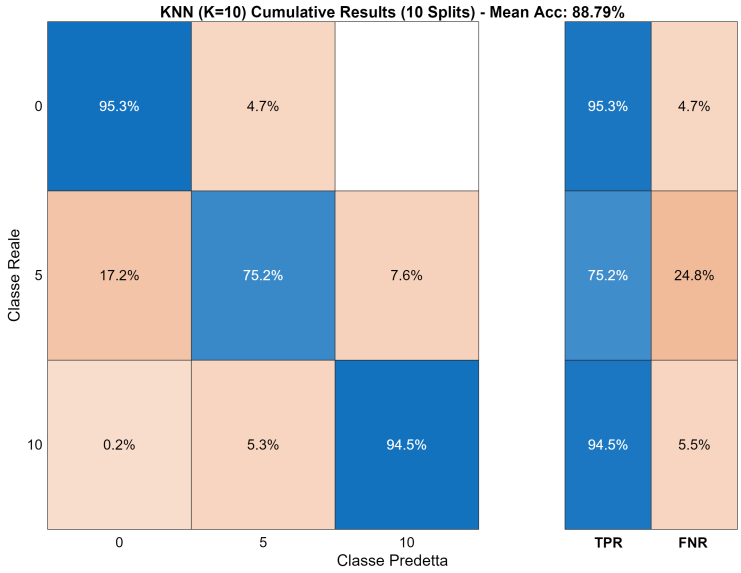


Figura 23: Confusion Matrix del modello Neural Network per tre classi (Set ANOVA).

A chiusura del capitolo sulla modellazione, viene presentata la matrice di confusione che illustra le prestazioni del modello Neural Network multi-classe (Figura 24) addestrato sfruttando il set di feature non parametrico (Kruskal-Wallis).

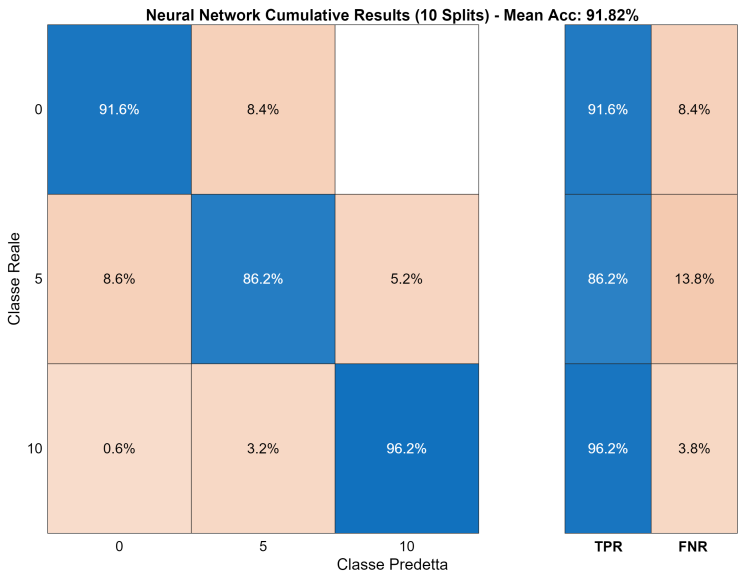


Figura 24: Confusion Matrix del modello Neural Network per tre classi (Set Kruskal-Wallis).

5 Risultati

In questa sezione vengono presentati e discussi i risultati finali ottenuti dai cinque classificatori analizzati: Neural Network (NN), Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbors (KNN) e Random Forest (RF).

Per valutare la robustezza dei modelli e l'efficacia delle due diverse tecniche di selezione delle feature esplorate, le prestazioni sono state misurate sia nello scenario di classificazione binaria sia in quello multi-classe (3 Classi). I valori riportati rappresentano l'accuratezza Media sui 10 split di validazione incrociata, accompagnata dalla relativa deviazione standard.

La Tabella 1 sintetizza numericamente tutte le configurazioni testate.

Classificatore	**Scenario**	**ANOVA**		**Kruskal-Wallis**	
		Media	**Dev. Std**	**Media**	**Dev. Std**
Neural Network	2 Classi	94.47%	±2.08%	94.55%	±1.51%
	3 Classi	91.29%	±3.47%	91.82%	±2.57%
SVM	2 Classi	91.74%	±2.20%	94.32%	±1.39%
	3 Classi	90.68%	±3.68%	91.74%	±1.69%
Decision Tree	2 Classi	92.20%	±1.60%	92.58%	±2.20%
	3 Classi	85.98%	±2.93%	86.89%	±3.01%
KNN	2 Classi	92.80%	±2.32%	93.18%	±1.68%
	3 Classi	90.68%	±2.48%	88.79%	±2.52%
Random Forest	2 Classi	94.24%	±1.64%	94.55%	±1.46%
	3 Classi	91.74%	±2.21%	91.74%	±2.59%

Tabella 1: Riassunto delle performance dei classificatori

5.1 Analisi per Due Classi

Il problema binario risulta essere più semplice per tutti i modelli, trattandosi della pura rilevazione della presenza di un'anomalia, indipendentemente dalla sua severità. Come si evince dalla Tabella 1, tutti i classificatori superano agevolmente la soglia del 91% di accuratezza.

In questo scenario, i risultati migliori e più stabili sono stati ottenuti dalla **Rete Neurale** e dalla **Random Forest**. Entrambi i modelli raggiungono un picco del

94.55% quando addestrati con le feature estratte con Kruskal-Wallis, presentando deviazioni standard contenute ($\approx 1.5\%$), a dimostrazione di un'eccellente capacità di generalizzazione e di una bassa incidenza di falsi allarmi o guasti mancati. Anche la SVM mostra un miglioramento notevole ($+2.58\%$) passando dalle feature ANOVA a quelle Kruskal.

5.2 Analisi per Tre Classi

Il passaggio alla classificazione multi-classe (Sano, Guasto 5%, Guasto 10%) introduce una complessità maggiore. La necessità di distinguere la soglia di severità del danno porta a un calo contenuto e generalizzato delle performance rispetto allo scenario binario.

Il modello che soffre maggiormente è il **Decision Tree**, la cui accuratezza diminuisce intorno all'86% $\sim$ 87%. Questo calo evidenzia i limiti di un singolo albero nel delineare confini decisionali complessi senza incorrere in errori di sovrapposizione tra i guasti 5% e quelli 10%.

Viceversa, i modelli non lineari più complessi confermano la loro superiorità, la **Rete Neurale** risulta il miglior classificatore in assoluto per il task a 3 classi, raggiungendo il 91.82% (Kruskal-Wallis), seguita da SVM e Random Forest (entrambi al 91.74%).

5.3 Confronto tra ANOVA e Kruskal-Wallis

Un risultato importante emerso da queste prove riguarda l'efficacia dei due diversi metodi di Feature Selection. Nella quasi totalità degli scenari testati, l'impiego delle feature estratte tramite il test non parametrico di **Kruskal-Wallis** ha garantito risultati superiori o equivalenti rispetto a quelli ottenuti con l'ANOVA.

Il beneficio più evidente dell'uso di Kruskal-Wallis si riscontra nella riduzione della **Deviazione Standard**. Modelli come la SVM (su 2 e 3 classi) e la Rete Neurale (su 3 classi) mostrano una sensibile diminuzione della varianza dell'errore. Questo suggerisce che le feature selezionate tramite il confronto delle mediane (Kruskal) risultano intrinsecamente più robuste e stabili rispetto agli outlier, facilitando la convergenza degli algoritmi e rendendo le performance predittive meno dipendenti dalla specifica suddivisione dei dati di addestramento. L'unica eccezione rilevante è rappresentata dal KNN nel task a 3 classi, dove ANOVA ha performato meglio.

6 Explainability

Nel contesto della diagnostica e della manutenzione predittiva, ottenere un'elevata accuratezza predittiva non è sufficiente. È fondamentale comprendere le motivazioni alla base di un allarme generato dall'algoritmo per poter intervenire tempestivamente e con cognizione di causa.

Per questo motivo, l'analisi predittiva è stata affiancata da uno studio approfondito di *eXplainable*. Poiché i modelli utilizzati spaziano da algoritmi intrinsecamente interpretabili (white-box) a modelli complessi e opachi (black-box), sono state adottate tecniche di interpretazione specifiche e differenziate per ciascun classificatore.

6.1 Decision Tree: Interpretazione Diretta e Rule Extraction

Il Decision Tree è un modello white-box, le sue decisioni sono una semplice sequenza di scelte `if-then-else`. L'explainability per questo algoritmo non richiede approssimazioni o permutazioni, ma si basa sulla **Rule Extraction** (estrazione delle regole).

Nella (Figura 34) è riportato l'albero decisionale. E' stata generata una *Tabella di Regole* organizzata con la seguente logica: "Identificativo Regola", "Condizione da Verificare", "Azione se Vero", "Azione se Falso".

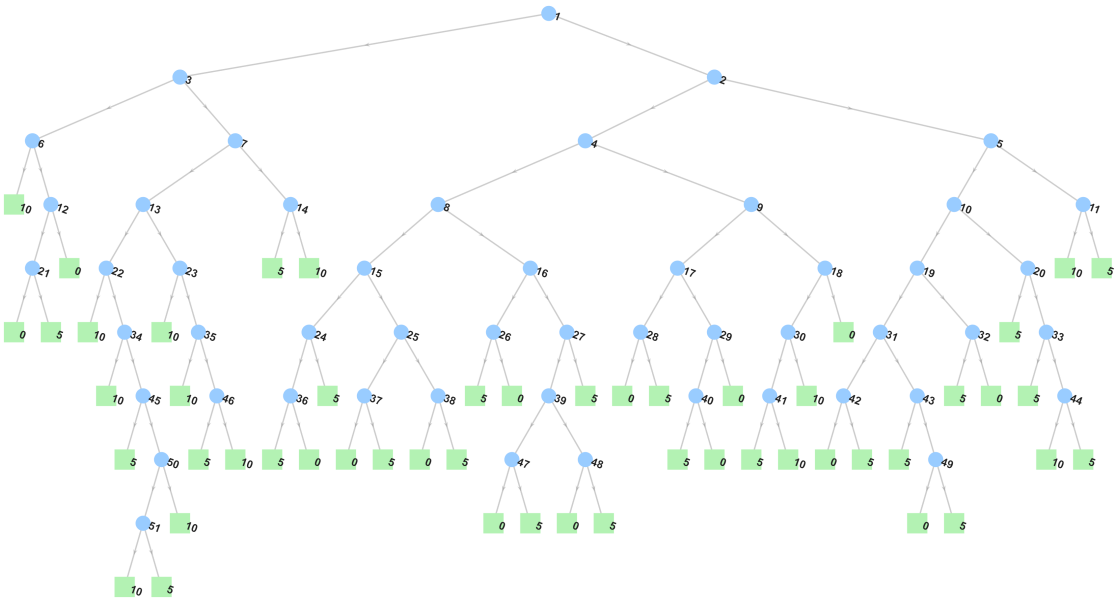


Figura 25: Decision Tree

Tabella 2: Tabella di navigazione delle regole per il Decision Tree

Regola	Condizione	Se VERO	Se FALSO
1	`FRM_1/vibe_y_sigstats/PeakValue < 1.7935`	Vai a Regola 2	Vai a Regola 3
2	`FRM_1/vibe_x_sigstats/PeakValue < 1.9051`	Vai a Regola 4	Vai a Regola 5
3	`FRM_1/vibe_y_sigstats/RMS < 1.5515`	Vai a Regola 6	Vai a Regola 7
4	`FRM_1/vibe_y_sigstats/PeakValue < 1.4227`	Vai a Regola 8	Vai a Regola 9
5	`FRM_1/vibe_y_sigstats/RMS < 1.5310`	Vai a Regola 10	Vai a Regola 11
6	`FRM_1/vibe_x_sigstats/RMS < 1.8702`	Vai a Regola 12	CLASSE: 10
7	`FRM_1/acc_x_sigstats/Mean < 0.2174`	Vai a Regola 13	Vai a Regola 14

Continua nella pagina successiva...

Tabella 2 – continua dalla pagina precedente

Regola	Condizione	Se VERO	Se FALSO
8	FRM_1/acc_x_sigstats/Mean < 0.0992	Vai a Regola 15	Vai a Regola 16
9	FRM_1/vibe_x_sigstats/PeakValue < 1.6377	Vai a Regola 17	Vai a Regola 18
10	FRM_1/vibe_x_sigstats/RMS < 1.6352	Vai a Regola 19	Vai a Regola 20
11	FRM_1/acc_x_sigstats/Mean < -0.0828	CLASSE: 10	CLASSE: 5
12	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0921	Vai a Regola 21	CLASSE: 0
13	FRM_1/acc_x_sigstats/Mean < -0.0928	Vai a Regola 22	Vai a Regola 23
14	FRM_1/pwm_5_sigstats/Std < 56.9447	CLASSE: 5	CLASSE: 10
15	FRM_1/vibe_y_sigstats/PeakValue < 1.2542	Vai a Regola 24	Vai a Regola 25
16	FRM_1/vibe_y_sigstats/RMS < 0.7830	Vai a Regola 26	Vai a Regola 27
17	FRM_1/vibe_z_sigstats/Mean < 1.0858	Vai a Regola 28	Vai a Regola 29
18	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0619	Vai a Regola 30	CLASSE: 0
19	FRM_1/vibe_y_sigstats/PeakValue < 1.3063	Vai a Regola 31	Vai a Regola 32
20	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0377	CLASSE: 5	Vai a Regola 33
21	FRM_1/vibe_x_sigstats/PeakValue < 1.4066	CLASSE: 0	CLASSE: 5
22	FRM_1/acc_x_sigstats/Mean < -0.2574	CLASSE: 10	Vai a Regola 34
23	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0359	CLASSE: 10	Vai a Regola 35
24	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0010	Vai a Regola 36	CLASSE: 5
25	FRM_1/pwm_5_sigstats/Std < 27.9306	Vai a Regola 37	Vai a Regola 38
26	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0517	CLASSE: 5	CLASSE: 0
27	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0004	Vai a Regola 39	CLASSE: 5
28	FRM_1/acc_x_sigstats/Mean < 0.2421	CLASSE: 0	CLASSE: 5
29	FRM_1/vibe_z_sigstats/PeakValue < 1.4850	Vai a Regola 40	CLASSE: 0
30	FRM_1/vibe_x_sigstats/RMS < 1.5832	Vai a Regola 41	CLASSE: 10
31	FRM_1/pwm_1_sigstats/Std < 19.1363	Vai a Regola 42	Vai a Regola 43
32	FRM_1/pwm_5_sigstats/Std < 41.5530	CLASSE: 5	CLASSE: 0
33	FRM_1/vibe_y_sigstats/RMS < 1.4110	CLASSE: 5	Vai a Regola 44
34	FRM_1/vibe_z_sigstats/Mean < 1.2414	CLASSE: 10	Vai a Regola 45
35	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0864	CLASSE: 10	Vai a Regola 46
36	FRM_1/vibe_z_sigstats/Mean < 0.6899	CLASSE: 5	CLASSE: 0
37	FRM_1/vibe_x_sigstats/PeakValue < 1.6311	CLASSE: 0	CLASSE: 5
38	FRM_1/vibe_y_sigstats/Mean < 1.1921	CLASSE: 0	CLASSE: 5
39	FRM_1/vibe_y_sigstats/Mean < 0.9069	Vai a Regola 47	Vai a Regola 48
40	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0481	CLASSE: 5	CLASSE: 0
41	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0008	CLASSE: 5	CLASSE: 10
42	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0313	CLASSE: 0	CLASSE: 5
43	FRM_1/vibe_y_sigstats/Mean < 0.7197	CLASSE: 5	Vai a Regola 49
44	FRM_1/acc_x_sigstats/Mean < -0.1103	CLASSE: 10	CLASSE: 5
45	FRM_1/vibe_z_sigstats/RMS < 1.2552	CLASSE: 5	Vai a Regola 50
46	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0381	CLASSE: 5	CLASSE: 10
47	FRM_1/pwm_1_sigstats/Std < 20.0449	CLASSE: 0	CLASSE: 5
48	FRM_1/vibe_x_sigstats/Mean < 1.3973	CLASSE: 0	CLASSE: 5
49	FRM_1/acc_x_sigstats/Mean < 0.0721	CLASSE: 0	CLASSE: 5
50	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0008	Vai a Regola 51	CLASSE: 10
51	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0007	CLASSE: 10	CLASSE: 5

La Tabella delle Regole (Tabella 2) mette in evidenza una struttura decisionale

fortemente guidata dalle vibrazioni:

- la radice e i primi livelli dell'albero utilizzano soglie su `vibe_y_sigstats/PeakValue`, `vibe_x_sigstats/PeakValue` e `vibe_y_sigstats/RMS` per discriminare tra condizioni chiaramente nominali e condizioni anomale;
- le foglie associate alla classe 0 sono spesso caratterizzate da valori di RMS e PeakValue inferiori a soglie precise;
- la distinzione tra classe 5% e 10% viene affinata più in profondità tramite mean di `acc_x`, RMS su `vibe_z` e indici di tipo statistico sui PWM, che riflettono lo sforzo di controllo aggiuntivo richiesto in presenza di usura più severa.

Il grafico dei vari percorsi decisionali facilita la validazione da parte dell'utente: una volta nota la regola, è possibile verificare direttamente sul grafico se il dato si colloca nella regione prevista per un certo stato di salute.

6.2 Reti Neurali (NN): Modello Surrogato

Le Reti Neurali Feedforward, a causa dei molteplici livelli nascosti e delle funzioni di attivazione non lineari, sono delle black box. Per estrarre una spiegazione logica dal modello, è stato utilizzato un **Modello Surrogato**.

La tecnica del modello surrogato consiste nell'addestrare un modello interpretabile (in questo caso, un Albero Decisionale) non sulle etichette reali del dataset, ma *sulle predizioni generate dalla Rete Neurale*. L'obiettivo dell'albero non è diagnosticare il guasto, bensì imitare il comportamento della rete. Per massimizzare questa imitazione, l'albero surrogato è stato configurato senza alcun limite di profondità o potatura (`'Prune', 'off'`).

L'efficacia di questa tecnica si misura tramite la **Fidelity (Fedeltà)**, che rappresenta la percentuale di accuratezza tra le risposte della Rete Neurale e le risposte del Surrogato. Come mostrato nel grafico a torta (Figura 26), il modello surrogato riesce a replicare quasi perfettamente la logica della rete. Nella (Figura 27) è riportato il Decision Tree usato come modello surrogato.



Figura 26: Fedeltà del Modello Surrogato

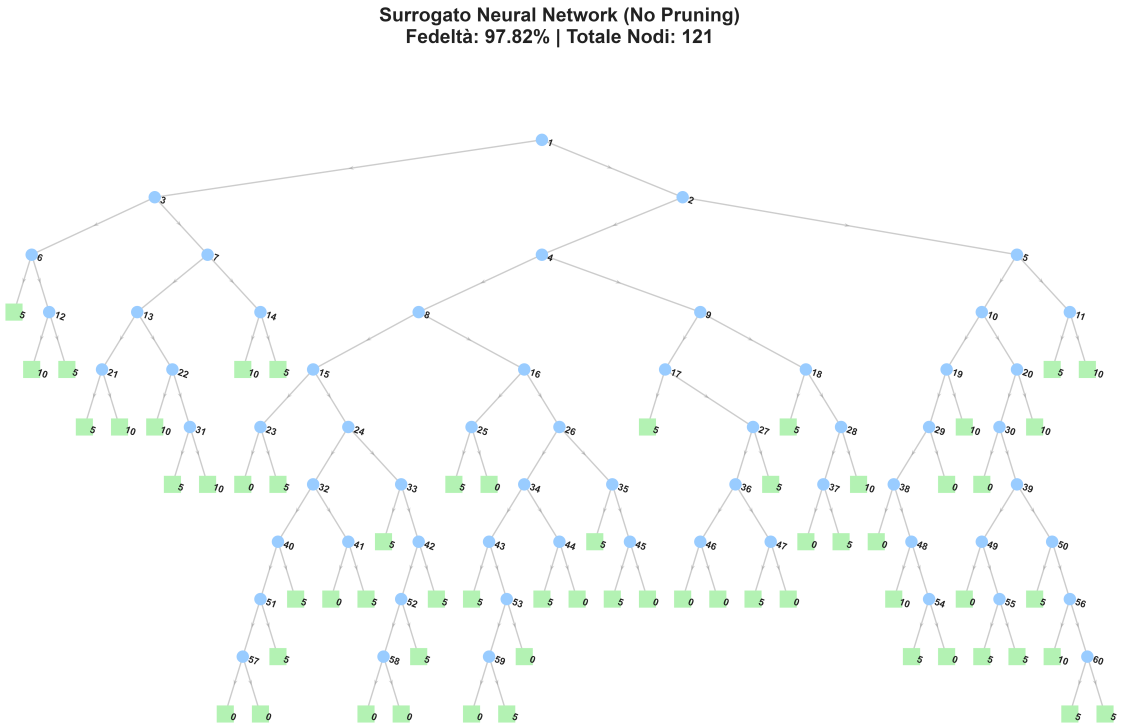


Figura 27: Rappresentazione grafica dell'Albero Surrogato

Tabella 3: Tabella di navigazione delle regole per il Modello Surrogato (Neural Network)

Regola	Condizione	Se VERO	Se FALSO
1	`FRM_1/vibe_y_sigstats/Mean < 1.5407`	Vai a Regola 2	Vai a Regola 3
2	`FRM_1/vibe_x_sigstats/RMS < 1.5435`	Vai a Regola 4	Vai a Regola 5
3	`FRM_1/vibe_y_sigstats/PeakValue < 1.8322`	Vai a Regola 6	Vai a Regola 7
4	`FRM_1/vibe_y_sigstats/Mean < 1.0945`	Vai a Regola 8	Vai a Regola 9
5	`FRM_1/vibe_y_sigstats/PeakValue < 1.8136`	Vai a Regola 10	Vai a Regola 11
6	`FRM_1/vibe_z_sigstats/Mean < 1.2171`	Vai a Regola 12	CLASSE: 5
7	`FRM_1/acc_x_sigstats/Mean < 0.2263`	Vai a Regola 13	Vai a Regola 14
8	`FRM_1/acc_x_sigstats/Mean < 0.0364`	Vai a Regola 15	Vai a Regola 16
9	`FRM_1/vibe_x_sigstats/PeakValue < 1.6503`	Vai a Regola 17	Vai a Regola 18
10	`FRM_1/acc_x_sigstats/Mean < -0.3047`	Vai a Regola 19	Vai a Regola 20
11	`FRM_1/vibe_x_sigstats/Mean < 1.8772`	CLASSE: 5	CLASSE: 10
12	`FRM_1/acc_x_sigstats/Mean < 0.1042`	CLASSE: 10	CLASSE: 5
13	`FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0279`	Vai a Regola 21	Vai a Regola 22
14	`FRM_1/pwm_2_sigstats/Std < 68.7983`	CLASSE: 10	CLASSE: 5
15	`FRM_1/pwm_5_sigstats/Std < 21.4424`	Vai a Regola 23	Vai a Regola 24

Continua nella pagina successiva...

Tabella 3 – continua dalla pagina precedente

Regola	Condizione	Se VERO	Se FALSO
16	FRM_1/vibe_y_sigstats/Mean < 0.7493	Vai a Regola 25	Vai a Regola 26
17	FRM_1/pwm_2_sigstats/Std < 24.4294	CLASSE: 5	Vai a Regola 27
18	FRM_1/pwm_5_sigstats/CrestFactor < 1.0514	CLASSE: 5	Vai a Regola 28
19	FRM_1/vibe_x_sigstats/RMS < 1.9124	Vai a Regola 29	CLASSE: 10
20	FRM_1/vibe_x_sigstats/PeakValue < 2.7177	Vai a Regola 30	CLASSE: 10
21	FRM_1/vibe_y_sigstats/PeakValue < 1.8706	CLASSE: 5	CLASSE: 10
22	FRM_1/acc_x_sigstats/Mean < 0.1113	CLASSE: 10	Vai a Regola 31
23	FRM_1/acc_x_sigstats/Mean < -0.2299	CLASSE: 0	CLASSE: 5
24	FRM_1/pwm_5_sigstats/Std < 27.7416	Vai a Regola 32	Vai a Regola 33
25	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0526	CLASSE: 5	CLASSE: 0
26	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0004	Vai a Regola 34	Vai a Regola 35
27	FRM_1/acc_x_sigstats/Mean < 0.1920	Vai a Regola 36	CLASSE: 5
28	FRM_1/vibe_x_sigstats/Mean < 1.4381	Vai a Regola 37	CLASSE: 10
29	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0507	Vai a Regola 38	CLASSE: 0
30	FRM_1/vibe_z_sigstats/PeakValue < 0.8230	CLASSE: 0	Vai a Regola 39
31	FRM_1/acc_x_sigstats/Mean < 0.1134	CLASSE: 5	CLASSE: 10
32	FRM_1/vibe_y_sigstats/PeakValue < 1.2498	Vai a Regola 40	Vai a Regola 41
33	FRM_1/pwm_1_sigstats/Std < 17.7403	CLASSE: 5	Vai a Regola 42
34	FRM_1/vibe_x_sigstats/PeakValue < 1.7386	Vai a Regola 43	Vai a Regola 44
35	FRM_1/vibe_x_sigstats/PeakValue < 1.5629	CLASSE: 5	Vai a Regola 45
36	FRM_1/vibe_x_sigstats/PeakValue < 1.5420	Vai a Regola 46	Vai a Regola 47
37	FRM_1/pwm_1_sigstats/Std < 37.3078	CLASSE: 0	CLASSE: 5
38	FRM_1/vibe_x_sigstats/PeakValue < 1.9165	CLASSE: 0	Vai a Regola 48
39	FRM_1/vibe_x_sigstats/RMS < 1.6302	Vai a Regola 49	Vai a Regola 50
40	FRM_1/acc_x_sigstats/Mean < 0.0196	Vai a Regola 51	CLASSE: 5
41	FRM_1/vibe_x_sigstats/PeakValue < 1.6073	CLASSE: 0	CLASSE: 5
42	FRM_1/vibe_z_sigstats/PeakValue < 1.9878	Vai a Regola 52	CLASSE: 5
43	FRM_1/acc_x_sigstats/Mean < 0.0378	CLASSE: 5	Vai a Regola 53
44	FRM_1/vibe_x_sigstats/Mean < 1.4037	CLASSE: 5	CLASSE: 0
45	FRM_1/vibe_z_sigstats/Mean < 0.9338	CLASSE: 5	CLASSE: 0
46	FRM_1/acc_x_sigstats/Mean < -0.5351	CLASSE: 0	CLASSE: 0
47	FRM_1/vibe_z_sigstats/PeakValue < 1.4542	CLASSE: 5	CLASSE: 0
48	FRM_1/vibe_z_sigstats/PeakValue < 1.1080	CLASSE: 10	Vai a Regola 54
49	FRM_1/vibe_z_sigstats/Mean < 0.8796	CLASSE: 0	Vai a Regola 55
50	FRM_1/pwm_5_sigstats/CrestFactor < 1.0649	CLASSE: 5	Vai a Regola 56
51	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0013	Vai a Regola 57	CLASSE: 5
52	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0013	Vai a Regola 58	CLASSE: 5
53	FRM_1/pwm_5_sigstats/Std < 31.0182	Vai a Regola 59	CLASSE: 0
54	FRM_1/vibe_z_sigstats/PeakValue < 1.9391	CLASSE: 5	CLASSE: 0
55	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0627	CLASSE: 5	CLASSE: 5
56	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0659	CLASSE: 10	Vai a Regola 60
57	FRM_1/pwm_2_sigstats/Std < 34.3005	CLASSE: 0	CLASSE: 0
58	FRM_1/vibe_y_sigstats/PeakValue < 1.5118	CLASSE: 0	CLASSE: 0
59	FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0290	CLASSE: 0	CLASSE: 5
60	FRM_1/pwm_5_sigstats/Std < 38.3475	CLASSE: 5	CLASSE: 5

Dall'analisi della struttura dell'albero surrogato (Figura 27) emergono alcuni aspetti rilevanti:

- le prime scissioni si basano quasi esclusivamente su segnali come `vibe_y_sigstats` e `vibe_x_sigstats`, utilizzati per separare in blocco i casi sicuramente sani da quelli potenzialmente guasti;
- i livelli successivi introducono soglie su mean di `acc_x` e sugli indici statistici dei PWM, analogamente a quanto osservato nel Decision Tree "diretto";
- nelle foglie che corrispondono alla classe 10% il modello surrogato mostra condizioni congiunte di elevata vibrazione e incremento di dispersione sui comandi PWM.

Il fatto che il modello surrogato utilizzi pattern molto simili a quelli individuati dagli altri algoritmi white-box suggerisce che la rete neurale non stia apprendendo correlazioni spurie, ma stia invece sfruttando gli stessi meccanismi fisici di base in forma più ricca e non lineare.

6.3 Random Forest: OOB Importance e Analisi Locale (Gini Decrease)

Il Random Forest è un metodo *ensemble* costituito da decine di alberi decisionali. Sebbene il singolo albero sia trasparente, il meccanismo di voto a maggioranza sull'intera foresta lo trasforma in un modello black-box. L'explainability è stata pertanto divisa in un'analisi globale e in una locale.

Analisi Globale (OOB Importance): Sfruttando la natura intrinseca del *Bagging*, il Random Forest è in grado di autovalutarsi durante l'addestramento. Ogni albero viene addestrato su un sottoinsieme dei dati, lasciando fuori alcuni campioni noti come *Out-of-Bag (OOB)*. L'importanza di ogni feature viene calcolata permutando i valori proprio su questi campioni esclusi (OOB) e misurando l'incremento dell'errore di classificazione. Questa metrica è altamente affidabile in quanto calcolata su dati "mai visti" dal singolo albero, senza richiedere un test set separato (Figura 28).

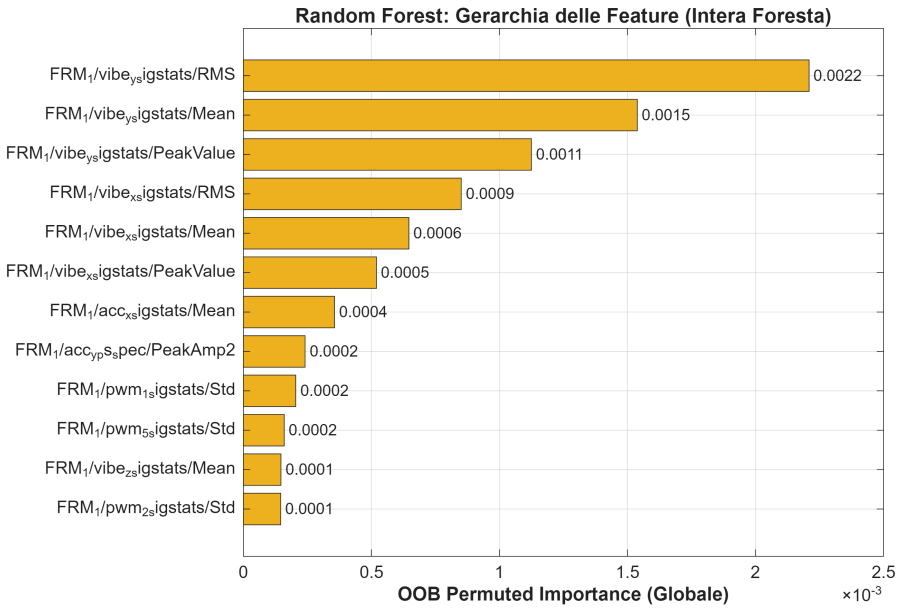


Figura 28: Global Feature Importance (OOB) dell'intera foresta Random Forest.

Analisi Locale (Gini Decrease): Per capire nel dettaglio come le feature operano nello smistamento, l'algoritmo di spiegazione esplora la foresta ed estrae due alberi rappresentativi (Albero A e Albero B). Per questi alberi, viene calcolata un'importanza locale basata sulla **Riduzione del Rischio (Gini Decrease)**. Questa metrica matematica valuta la purezza di un nodo: una feature ha un'importanza locale alta se, quando usata per dividere i dati in un ramo, riesce a

separare nettamente le classi. L'esportazione grafica dell'albero e dei relativi pesi (Figura 30) chiarisce come la foresta diversifichi le proprie decisioni.

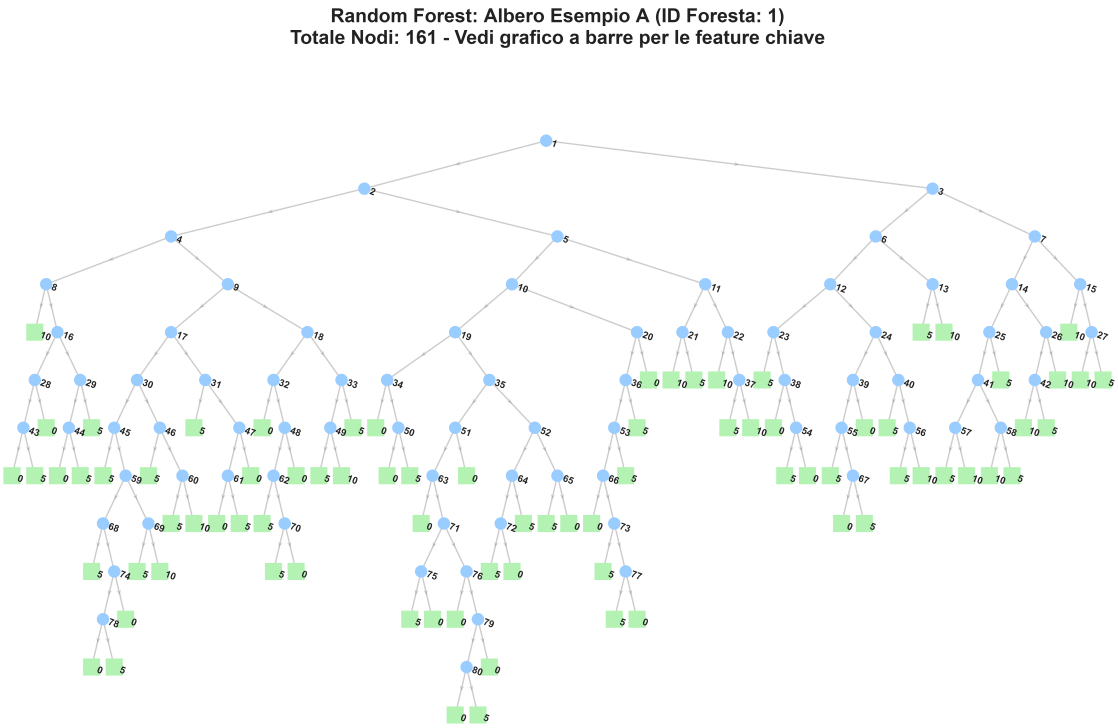


Figura 29: Albero A

Tabella 4: Tabella di navigazione delle regole per l'Albero A (Random Forest)

Regola	Condizione	Se VERO	Se FALSO
1	`FRM_1/vibe_x_sigstats/PeakValue < 1.9541`	Vai a Regola 2	Vai a Regola 3
2	`FRM_1/pwm_5_sigstats/Std < 31.5089`	Vai a Regola 4	Vai a Regola 5
3	`FRM_1/vibe_y_sigstats/PeakValue < 1.8136`	Vai a Regola 6	Vai a Regola 7
4	`FRM_1/vibe_x_sigstats/RMS < 1.2328`	Vai a Regola 8	Vai a Regola 9
5	`FRM_1/vibe_y_sigstats/Mean < 1.5477`	Vai a Regola 10	Vai a Regola 11
6	`FRM_1/vibe_y_sigstats/RMS < 1.4971`	Vai a Regola 12	Vai a Regola 13
7	`FRM_1/vibe_x_sigstats/Mean < 1.9043`	Vai a Regola 14	Vai a Regola 15
8	`FRM_1/vibe_y_sigstats/PeakValue < 1.8875`	Vai a Regola 16	CLASSE: 10
9	`FRM_1/vibe_x_sigstats/PeakValue < 1.7138`	Vai a Regola 17	Vai a Regola 18
10	`FRM_1/vibe_y_sigstats/Mean < 1.2062`	Vai a Regola 19	Vai a Regola 20
11	`FRM_1/pwm_2_sigstats/Std < 32.0303`	Vai a Regola 21	Vai a Regola 22

Continua nella pagina successiva...

Tabella 4 – continua dalla pagina precedente

Regola	Condizione	Se VERO	Se FALSO
12	FRM_1/vibe_x_sigstats/RMS < 1.5800	Vai a Regola 23	Vai a Regola 24
13	FRM_1/pwm_1_sigstats/Std < 21.6101	CLASSE: 5	CLASSE: 10
14	FRM_1/vibe_z_sigstats/RMS < 1.2617	Vai a Regola 25	Vai a Regola 26
15	FRM_1/pwm_1_sigstats/ImpulseFactor < 1.0732	CLASSE: 10	Vai a Regola 27
16	FRM_1/vibe_x_sigstats/PeakValue < 1.5139	Vai a Regola 28	Vai a Regola 29
17	FRM_1/vibe_z_sigstats/PeakValue < 1.4152	Vai a Regola 30	Vai a Regola 31
18	FRM_1/vibe_y_sigstats/RMS < 0.9764	Vai a Regola 32	Vai a Regola 33
19	FRM_1/vibe_y_sigstats/Mean < 0.7904	Vai a Regola 34	Vai a Regola 35
20	FRM_1/pwm_5_sigstats/ImpulseFactor < 1.0841	Vai a Regola 36	CLASSE: 0
21	FRM_1/vibe_z_sigstats/RMS < 1.3520	CLASSE: 10	CLASSE: 5
22	FRM_1/pwm_1_sigstats/CrestFactor < 1.0769	CLASSE: 10	Vai a Regola 37
23	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0415	CLASSE: 5	Vai a Regola 38
24	FRM_1/vibe_y_sigstats/Mean < 1.4068	Vai a Regola 39	Vai a Regola 40
25	FRM_1/pwm_1_sigstats/Std < 45.5891	Vai a Regola 41	CLASSE: 5
26	FRM_1/vibe_y_sigstats/PeakValue < 2.2559	Vai a Regola 42	CLASSE: 10
27	FRM_1/vibe_z_sigstats/Mean < 1.6591	CLASSE: 10	CLASSE: 5
28	FRM_1/vibe_z_sigstats/RMS < 0.9177	Vai a Regola 43	CLASSE: 0
29	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0353	Vai a Regola 44	CLASSE: 5
30	FRM_1/vibe_z_sigstats/RMS < 1.0464	Vai a Regola 45	Vai a Regola 46
31	FRM_1/pwm_1_sigstats/Std < 14.1263	CLASSE: 5	Vai a Regola 47
32	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0001	CLASSE: 0	Vai a Regola 48
33	FRM_1/acc_x_sigstats/Mean < -0.5053	Vai a Regola 49	CLASSE: 5
34	FRM_1/vibe_y_sigstats/Mean < 0.7117	CLASSE: 0	Vai a Regola 50
35	FRM_1/vibe_y_sigstats/RMS < 1.0934	Vai a Regola 51	Vai a Regola 52
36	FRM_1/pwm_1_sigstats/Std < 51.2333	Vai a Regola 53	CLASSE: 5
37	FRM_1/pwm_2_sigstats/Std < 102.1170	CLASSE: 5	CLASSE: 10
38	FRM_1/vibe_y_sigstats/Mean < 1.0411	CLASSE: 0	Vai a Regola 54
39	FRM_1/vibe_z_sigstats/PeakValue < 1.9087	Vai a Regola 55	CLASSE: 0
40	FRM_1/vibe_x_sigstats/RMS < 1.9113	CLASSE: 5	Vai a Regola 56
41	FRM_1/vibe_y_sigstats/PeakValue < 1.8332	Vai a Regola 57	Vai a Regola 58
42	FRM_1/vibe_z_sigstats/PeakValue < 1.7889	CLASSE: 10	CLASSE: 5
43	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0001	CLASSE: 0	CLASSE: 5
44	FRM_1/vibe_y_sigstats/Mean < 1.3131	CLASSE: 0	CLASSE: 5
45	FRM_1/vibe_y_sigstats/RMS < 0.7229	CLASSE: 5	Vai a Regola 59
46	FRM_1/pwm_1_sigstats/CrestFactor < 1.0341	CLASSE: 5	Vai a Regola 60
47	FRM_1/pwm_5_sigstats/Std < 26.5826	Vai a Regola 61	CLASSE: 0
48	FRM_1/vibe_x_sigstats/Mean < 1.4139	Vai a Regola 62	CLASSE: 0
49	FRM_1/vibe_y_sigstats/Mean < 1.4091	CLASSE: 5	CLASSE: 10
50	FRM_1/vibe_x_sigstats/Mean < 1.2100	CLASSE: 0	CLASSE: 5
51	FRM_1/pwm_5_sigstats/Std < 42.4981	Vai a Regola 63	CLASSE: 0
52	FRM_1/acc_x_sigstats/Mean < 0.0972	Vai a Regola 64	Vai a Regola 65
53	FRM_1/vibe_x_sigstats/RMS < 1.3810	Vai a Regola 66	CLASSE: 5
54	FRM_1/vibe_z_sigstats/Mean < 1.3698	CLASSE: 5	CLASSE: 0
55	FRM_1/pwm_1_sigstats/ImpulseFactor < 1.0582	CLASSE: 5	Vai a Regola 67
56	FRM_1/pwm_1_sigstats/CrestFactor < 1.0313	CLASSE: 5	CLASSE: 10

Continua nella pagina successiva...

Tabella 4 – continua dalla pagina precedente

Regola	Condizione	Se VERO	Se FALSO
57	`FRM_1/vibe_y_sigstats/RMS < 1.5661`	CLASSE: 5	CLASSE: 10
58	`FRM_1/pwm_1_sigstats/CrestFactor < 1.0679`	CLASSE: 10	CLASSE: 5
59	`FRM_1/pwm_1_sigstats/Std < 26.7973`	Vai a Regola 68	Vai a Regola 69
60	`FRM_1/vibe_y_sigstats/RMS < 1.4902`	CLASSE: 5	CLASSE: 10
61	`FRM_1/vibe_x_sigstats/Mean < 1.3168`	CLASSE: 0	CLASSE: 5
62	`FRM_1/vibe_z_sigstats/PeakValue < 1.2514`	CLASSE: 5	Vai a Regola 70
63	`FRM_1/pwm_5_sigstats/CrestFactor < 1.0502`	CLASSE: 0	Vai a Regola 71
64	`FRM_1/pwm_1_sigstats/ClearanceFactor < 1.0725`	Vai a Regola 72	CLASSE: 5
65	`FRM_1/vibe_y_sigstats/Mean < 1.1610`	CLASSE: 5	CLASSE: 0
66	`FRM_1/vibe_x_sigstats/PeakValue < 1.4405`	CLASSE: 0	Vai a Regola 73
67	`FRM_1/vibe_x_sigstats/Mean < 1.6628`	CLASSE: 0	CLASSE: 5
68	`FRM_1/pwm_5_sigstats/Std < 21.8091`	CLASSE: 5	Vai a Regola 74
69	`FRM_1/vibe_y_sigstats/Mean < 1.4165`	CLASSE: 5	CLASSE: 10
70	`FRM_1/acc_x_sigstats/Mean < -0.2767`	CLASSE: 5	CLASSE: 0
71	`FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0523`	Vai a Regola 75	Vai a Regola 76
72	`FRM_1/pwm_1_sigstats/ImpulseFactor < 1.0334`	CLASSE: 5	CLASSE: 0
73	`FRM_1/pwm_1_sigstats/ImpulseFactor < 1.0520`	CLASSE: 5	Vai a Regola 77
74	`FRM_1/vibe_z_sigstats/PeakValue < 0.9533`	Vai a Regola 78	CLASSE: 0
75	`FRM_1/vibe_x_sigstats/Mean < 1.2670`	CLASSE: 5	CLASSE: 0
76	`FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0003`	CLASSE: 0	Vai a Regola 79
77	`FRM_1/vibe_z_sigstats/RMS < 1.1933`	CLASSE: 5	CLASSE: 0
78	`FRM_1/pwm_1_sigstats/CrestFactor < 1.0272`	CLASSE: 0	CLASSE: 5
79	`FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0004`	Vai a Regola 80	CLASSE: 0
80	`FRM_1/vibe_x_sigstats/RMS < 1.2074`	CLASSE: 0	CLASSE: 5

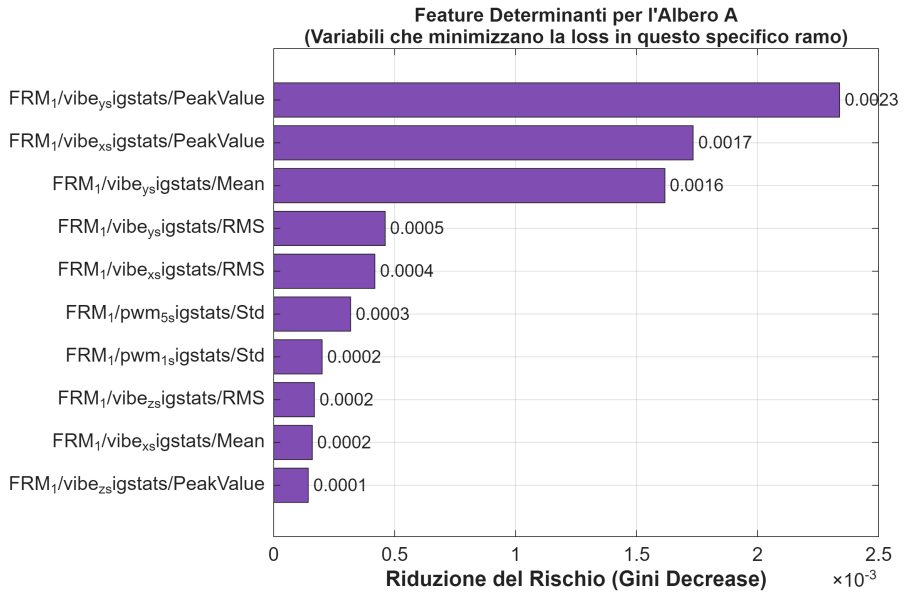


Figura 30: Importanza locale delle feature per il singolo Albero A, basata sulla metrica di Gini Decrease

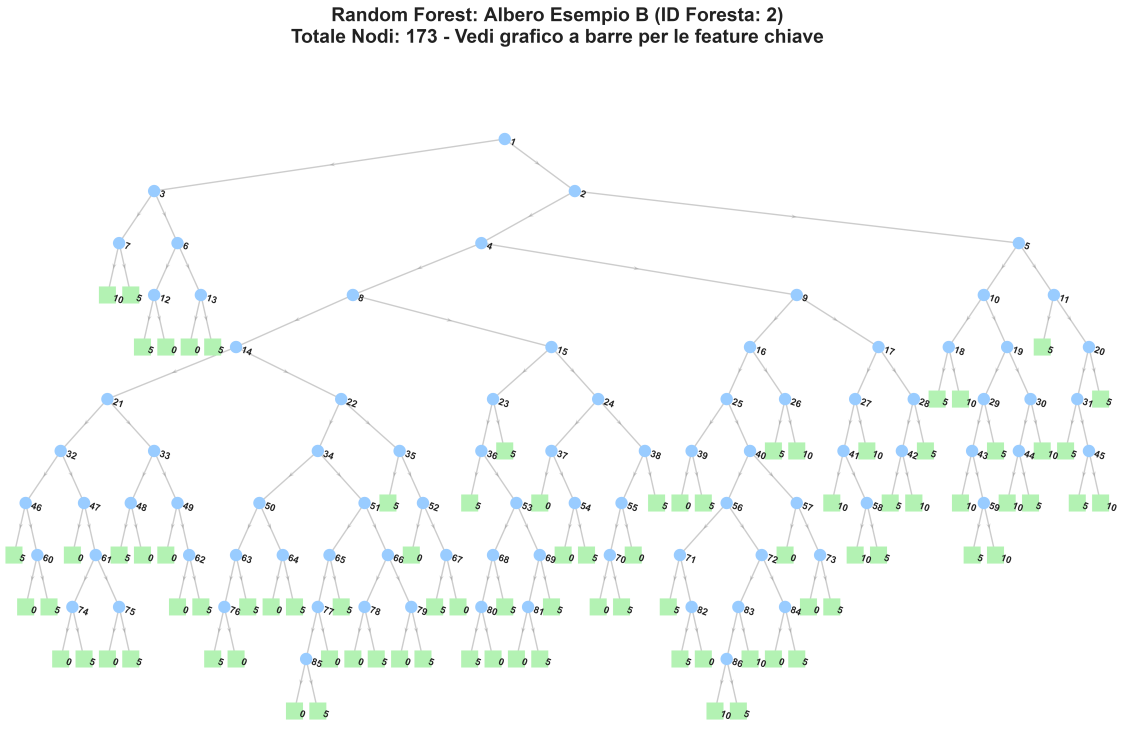


Figura 31: Albero B

Tabella 5: Tabella di navigazione delle regole per l'Albero B (Random Forest)

Regola	Condizione	Se VERO	Se FALSO
1	FRM_1/vibe_z_sigstats/Mean < 1.4200	Vai a Regola 2	Vai a Regola 3
2	FRM_1/vibe_y_sigstats/RMS < 1.5491	Vai a Regola 4	Vai a Regola 5
3	FRM_1/vibe_y_sigstats/Mean < 1.6399	Vai a Regola 6	Vai a Regola 7
4	FRM_1/vibe_x_sigstats/RMS < 1.5036	Vai a Regola 8	Vai a Regola 9
5	FRM_1/vibe_z_sigstats/Mean < 1.3359	Vai a Regola 10	Vai a Regola 11
6	FRM_1/vibe_x_sigstats/Mean < 1.3211	Vai a Regola 12	Vai a Regola 13
7	FRM_1/acc_x_sigstats/Mean < 0.2965	CLASSE: 10	CLASSE: 5
8	FRM_1/vibe_y_sigstats/RMS < 1.0965	Vai a Regola 14	Vai a Regola 15
9	FRM_1/vibe_y_sigstats/PeakValue < 1.7159	Vai a Regola 16	Vai a Regola 17
10	FRM_1/pwm_1_sigstats/Std < 19.0817	Vai a Regola 18	Vai a Regola 19
11	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0421	CLASSE: 5	Vai a Regola 20
12	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0002	CLASSE: 5	CLASSE: 0
13	FRM_1/vibe_y_sigstats/RMS < 0.9690	CLASSE: 0	CLASSE: 5
14	FRM_1/acc_x_sigstats/Mean < 0.0348	Vai a Regola 21	Vai a Regola 22
15	FRM_1/pwm_5_sigstats/ImpulseFactor < 1.0497	Vai a Regola 23	Vai a Regola 24
16	FRM_1/vibe_y_sigstats/Mean < 1.4657	Vai a Regola 25	Vai a Regola 26
17	FRM_1/pwm_2_sigstats/Std < 55.8300	Vai a Regola 27	Vai a Regola 28
18	FRM_1/vibe_x_sigstats/PeakValue < 2.2133	CLASSE: 5	CLASSE: 10
19	FRM_1/pwm_5_sigstats/ImpulseFactor < 1.0425	Vai a Regola 29	Vai a Regola 30
20	FRM_1/acc_x_sigstats/Mean < 0.2192	Vai a Regola 31	CLASSE: 5
21	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0387	Vai a Regola 32	Vai a Regola 33
22	FRM_1/vibe_x_sigstats/RMS < 1.3555	Vai a Regola 34	Vai a Regola 35
23	FRM_1/vibe_x_sigstats/Mean < 1.3163	Vai a Regola 36	CLASSE: 5
24	FRM_1/pwm_5_sigstats/CrestFactor < 1.0546	Vai a Regola 37	Vai a Regola 38
25	FRM_1/vibe_x_sigstats/PeakValue < 1.8550	Vai a Regola 39	Vai a Regola 40
26	FRM_1/vibe_x_sigstats/RMS < 1.8455	CLASSE: 5	CLASSE: 10
27	FRM_1/vibe_x_sigstats/Mean < 1.9071	Vai a Regola 41	CLASSE: 10
28	FRM_1/acc_x_sigstats/Mean < 0.0075	Vai a Regola 42	CLASSE: 5
29	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0421	Vai a Regola 43	CLASSE: 5
30	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0002	Vai a Regola 44	CLASSE: 10
31	FRM_1/pwm_1_sigstats/CrestFactor < 1.0293	CLASSE: 5	Vai a Regola 45
32	FRM_1/pwm_5_sigstats/Std < 23.4049	Vai a Regola 46	Vai a Regola 47
33	FRM_1/pwm_1_sigstats/Std < 16.4567	Vai a Regola 48	Vai a Regola 49
34	FRM_1/vibe_y_sigstats/RMS < 0.8410	Vai a Regola 50	Vai a Regola 51
35	FRM_1/vibe_y_sigstats/RMS < 0.7526	CLASSE: 5	Vai a Regola 52
36	FRM_1/pwm_2_sigstats/Std < 24.8711	CLASSE: 5	Vai a Regola 53
37	FRM_1/pwm_1_sigstats/CrestFactor < 1.0377	CLASSE: 0	Vai a Regola 54
38	FRM_1/acc_x_sigstats/Mean < 0.2267	Vai a Regola 55	CLASSE: 5
39	FRM_1/vibe_z_sigstats/Mean < 1.2280	CLASSE: 0	CLASSE: 5
40	FRM_1/pwm_1_sigstats/CrestFactor < 1.0564	Vai a Regola 56	Vai a Regola 57
41	FRM_1/acc_x_sigstats/Mean < -0.5226	CLASSE: 10	Vai a Regola 58
42	FRM_1/vibe_z_sigstats/Mean < 0.9945	CLASSE: 5	CLASSE: 10
43	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0005	CLASSE: 10	Vai a Regola 59

Continua nella pagina successiva...

Tabella 5 – continua dalla pagina precedente

Regola	Condizione	Se VERO	Se FALSO
44	FRM_1/pwm_5_sigstats/CrestFactor < 1.0549	CLASSE: 10	CLASSE: 5
45	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0003	CLASSE: 5	CLASSE: 10
46	FRM_1/pwm_1_sigstats/CrestFactor < 1.0244	CLASSE: 5	Vai a Regola 60
47	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0348	CLASSE: 0	Vai a Regola 61
48	FRM_1/pwm_1_sigstats/ImpulseFactor < 1.0293	CLASSE: 5	CLASSE: 0
49	FRM_1/vibe_z_sigstats/RMS < 1.1708	CLASSE: 0	Vai a Regola 62
50	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0561	Vai a Regola 63	Vai a Regola 64
51	FRM_1/acc_x_sigstats/Mean < 0.1155	Vai a Regola 65	Vai a Regola 66
52	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0001	CLASSE: 0	Vai a Regola 67
53	FRM_1/pwm_1_sigstats/CrestFactor < 1.0288	Vai a Regola 68	Vai a Regola 69
54	FRM_1/pwm_1_sigstats/Std < 32.2121	CLASSE: 0	CLASSE: 5
55	FRM_1/vibe_z_sigstats/RMS < 0.8737	Vai a Regola 70	CLASSE: 0
56	FRM_1/pwm_5_sigstats/Std < 39.0921	Vai a Regola 71	Vai a Regola 72
57	FRM_1/vibe_x_sigstats/RMS < 1.6008	CLASSE: 0	Vai a Regola 73
58	FRM_1/vibe_y_sigstats/Mean < 1.3180	CLASSE: 10	CLASSE: 5
59	FRM_1/vibe_y_sigstats/Mean < 1.6059	CLASSE: 5	CLASSE: 10
60	FRM_1/vibe_x_sigstats/PeakValue < 1.7296	CLASSE: 0	CLASSE: 5
61	FRM_1/pwm_5_sigstats/ImpulseFactor < 1.0366	Vai a Regola 74	Vai a Regola 75
62	FRM_1/vibe_y_sigstats/PeakValue < 1.4648	CLASSE: 0	CLASSE: 5
63	FRM_1/vibe_z_sigstats/PeakValue < 0.8697	Vai a Regola 76	CLASSE: 5
64	FRM_1/vibe_y_sigstats/RMS < 0.7867	CLASSE: 0	CLASSE: 5
65	FRM_1/pwm_1_sigstats/Std < 32.8850	Vai a Regola 77	CLASSE: 5
66	FRM_1/vibe_y_sigstats/PeakValue < 1.2641	Vai a Regola 78	Vai a Regola 79
67	FRM_1/pwm_5_sigstats/ClearanceFactor < 1.0513	CLASSE: 5	CLASSE: 0
68	FRM_1/pwm_5_sigstats/CrestFactor < 1.0422	Vai a Regola 80	CLASSE: 5
69	FRM_1/vibe_y_sigstats/PeakValue < 1.4156	Vai a Regola 81	CLASSE: 5
70	FRM_1/vibe_y_sigstats/PeakValue < 1.7962	CLASSE: 0	CLASSE: 5
71	FRM_1/vibe_z_sigstats/PeakValue < 1.5875	CLASSE: 5	Vai a Regola 82
72	FRM_1/acc_x_sigstats/Mean < -0.2108	Vai a Regola 83	Vai a Regola 84
73	FRM_1/acc_x_sigstats/Mean < -0.0785	CLASSE: 0	CLASSE: 5
74	FRM_1/acc_y_ps_spec/PeakAmp2 < 0.0010	CLASSE: 0	CLASSE: 5
75	FRM_1/vibe_z_sigstats/RMS < 1.2407	CLASSE: 0	CLASSE: 5
76	FRM_1/vibe_z_sigstats/Mean < 0.7522	CLASSE: 5	CLASSE: 0
77	FRM_1/pwm_1_sigstats/Std < 29.0259	Vai a Regola 85	CLASSE: 0
78	FRM_1/vibe_x_sigstats/PeakValue < 1.5406	CLASSE: 0	CLASSE: 5
79	FRM_1/acc_x_sigstats/Mean < 0.2204	CLASSE: 0	CLASSE: 5
80	FRM_1/vibe_y_sigstats/Mean < 1.1095	CLASSE: 5	CLASSE: 0
81	FRM_1/vibe_x_sigstats/PeakValue < 1.5900	CLASSE: 0	CLASSE: 5
82	FRM_1/pwm_5_sigstats/ImpulseFactor < 1.0479	CLASSE: 5	CLASSE: 0
83	FRM_1/vibe_y_sigstats/PeakValue < 1.6116	Vai a Regola 86	CLASSE: 10
84	FRM_1/vibe_z_sigstats/Mean < 0.7439	CLASSE: 0	CLASSE: 5
85	FRM_1/vibe_x_sigstats/RMS < 1.0966	CLASSE: 0	CLASSE: 5
86	FRM_1/vibe_x_sigstats/Mean < 2.0963	CLASSE: 10	CLASSE: 5

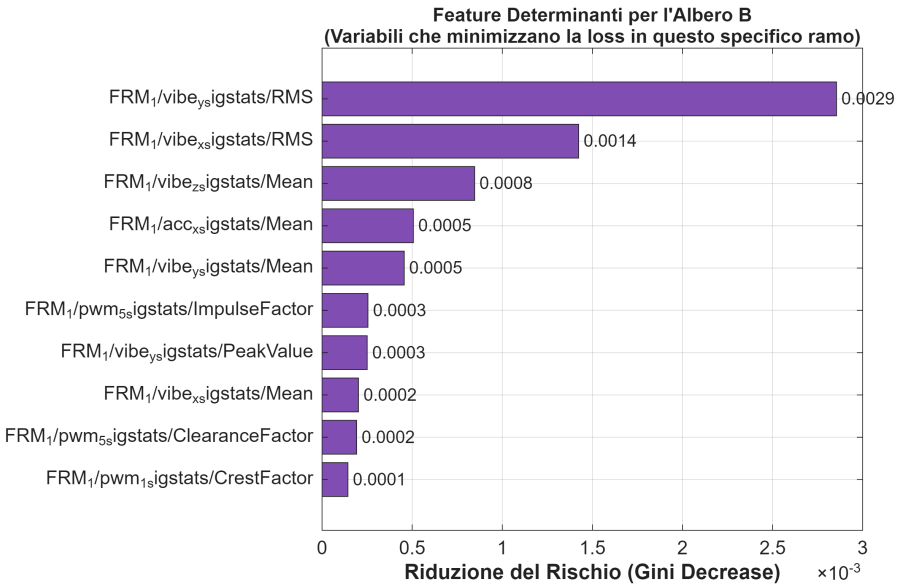


Figura 32: Importanza locale delle feature per il singolo Albero B, basata sulla metrica di Gini Decrease

Per illustrare la variabilità strutturale all'interno dell'ensemble, sono stati riportati due alberi di esempio con radici differenti (Figure 29 e 31), il primo albero utilizza come radice una feature vibrazionale, mentre il secondo impiega una metrica sui comandi PWM.
Il confronto tra i due grafici di *Gini Decrease* (Figure 30 e 32) mostra come, pur nelle differenze strutturali degli alberi, la distribuzione di importanza delle feature rimanga coerente: le metriche vibrazionali su `vibe_x`, `vibe_y`, `vibe_z` e alcune grandezze PWM dominano sistematicamente la riduzione dell'impurità.

6.4 SVM: Permutation Feature Importance

La SVM, specialmente quando utilizza kernel non lineari come quello polinomiale (adottato in questo progetto per mappare i dati in uno spazio a dimensionalità superiore), è un modello non interpretabile direttamente.

Per ovviare a questo problema, è stata implementata una tecnica di **Permutation Feature Importance**. Questa metodologia valuta l'importanza di una variabile misurando l'impatto che la sua assenza ha sulle prestazioni del modello. Il processo logico è il seguente:

1. Si calcola l'accuratezza *baseline* del modello addestrato sui dati originali del Test Set.

2. Si seleziona una singola feature e si permutano casualmente i suoi valori tra tutte le righe del dataset. Questa operazione rimuove qualsiasi legame logico tra quella variabile e la classe da predire, mantenendo inalterata la distribuzione statistica della feature stessa.

3. Si richiede al modello di effettuare nuovamente le predizioni sul dataset alterato e si registra il calo di accuratezza ($Acc_{baseline} - Acc_{permutata}$).

4. Se l'accuratezza diminuisce, la feature era cruciale per le decisioni della SVM. Se varia di poco, la feature era ridondante o ininfluente.

Per garantire robustezza statistica ed evitare fluttuazioni dovute alla casualità, il processo di permutazione è stato ripetuto 10 volte per ciascuna variabile. La Figura 33 mostra il ranking basato sul calo percentuale medio, permettendo di isolare visivamente le feature più discriminanti. Le barre indicano il calo percentuale di accuratezza quando la rispettiva feature viene resa casuale. Per semplicità è stata riportata solo le feature estratte con ANOVA applicato a tre classi.

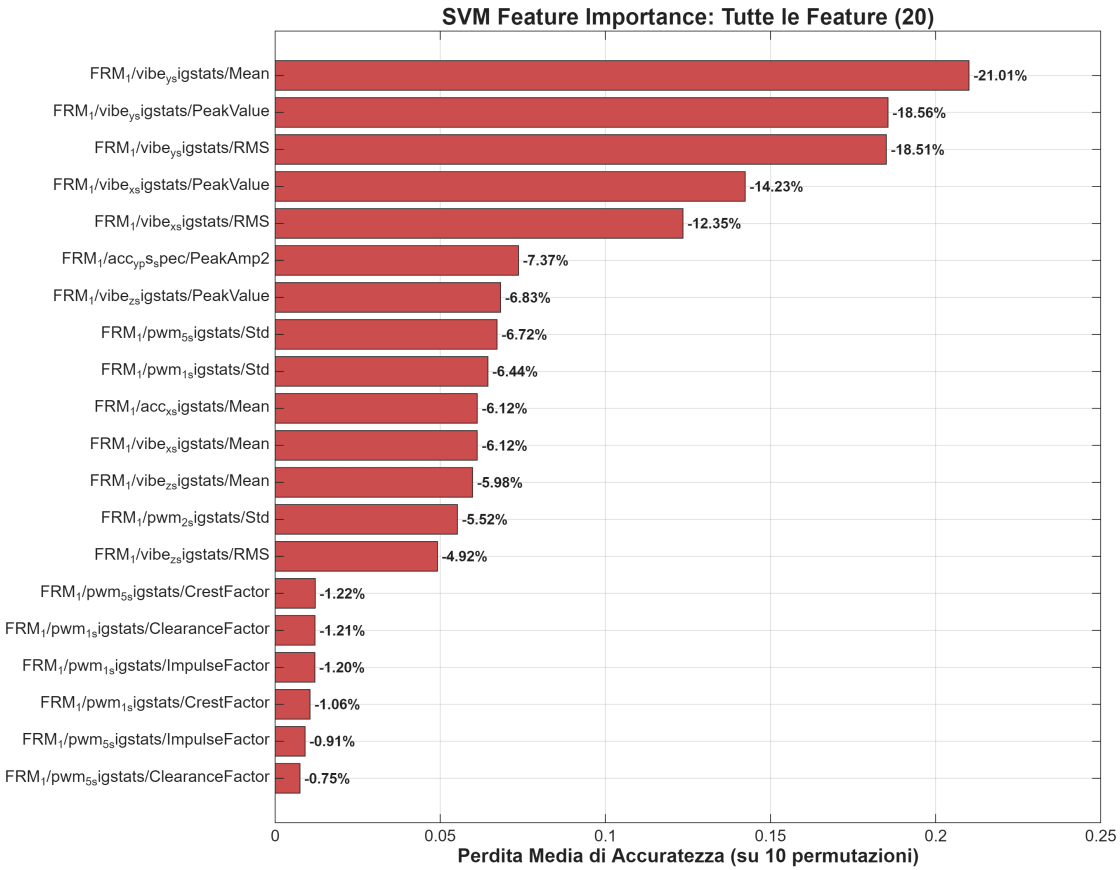


Figura 33: Permutation Feature Importance per il modello SVM

Nel grafico di importanza (Figura 33) si osserva come le feature derivate da `vibe_x` e `vibe_y` (in particolare RMS e PeakValue) occupino le prime posizioni del ranking.

La natura massimizzatrice del margine della SVM porta il modello a basarsi su un sottoinsieme relativamente compatto di feature fortemente discriminanti; questo si riflette in un grafico di importanza in cui poche variabili presentano un salto marcato rispetto alle altre.

6.5 K-Nearest Neighbors (KNN): Case Study Locale e Analisi dei Vicini

Il KNN è un modello basato sul concetto di apprendimento lazy (pigro), non crea veri confini decisionali matematici, ma memorizza i dati e classifica le nuove istanze in base alla distanza geometrica (*Distanza Euclidea*) rispetto ai campioni storici noti.

Poiché il KNN basa la sua predizione sul contesto spaziale locale, l'explainability adottata è di tipo *Local Case Study*. Scegliendo un volo specifico dal Test Set, l'algoritmo rintraccia i suoi 10 vicini più prossimi nello spazio delle feature. Per comprendere *perché* quel volo è stato classificato in un certo modo, vengono generati due grafici comparativi calcolando la distanza assoluta per singola feature ($|X_{query} - X_{neighbor}|$):

- **Analisi Intra-Classe (Miglior Vicino):** Viene calcolato lo scostamento tra il volo in esame e il suo vicino più prossimo appartenente alla sua stessa classe. Feature con valori molto bassi in questo grafico indicano le somiglianze che hanno portato alla predizione (Figura 34).
- **Analisi Inter-Classe (Vicino Nemico):** Viene individuato il punto più vicino appartenente a una classe *diversa* da quella predetta. Il grafico mostra su quali variabili i due voli differiscono maggiormente. Questo evidenzia quali feature abbiano agito in modo contrario, impedendo al modello di commettere un errore di classificazione (Figura 35).

Rango	ID Volo (Training)	Classe Appartenenza	Distanza Euclidea
1	251	0	0.9922
2	199	0	2.4421
3	513	5	2.5231
4	291	0	2.6908
5	22	0	2.8706
6	735	10	2.9086
7	314	0	3.0345
8	53	0	3.0860
9	387	5	3.6039
10	362	5	3.7320

Tabella 6: I 10 vicini più prossimi (Nearest Neighbors) individuati nel Training Set per il volo di test analizzato

E' stato scelto come esempio il volo con id 2. Entrambi i grafici condividono la stessa scala sull'asse X per permettere un confronto visivo ottimale.

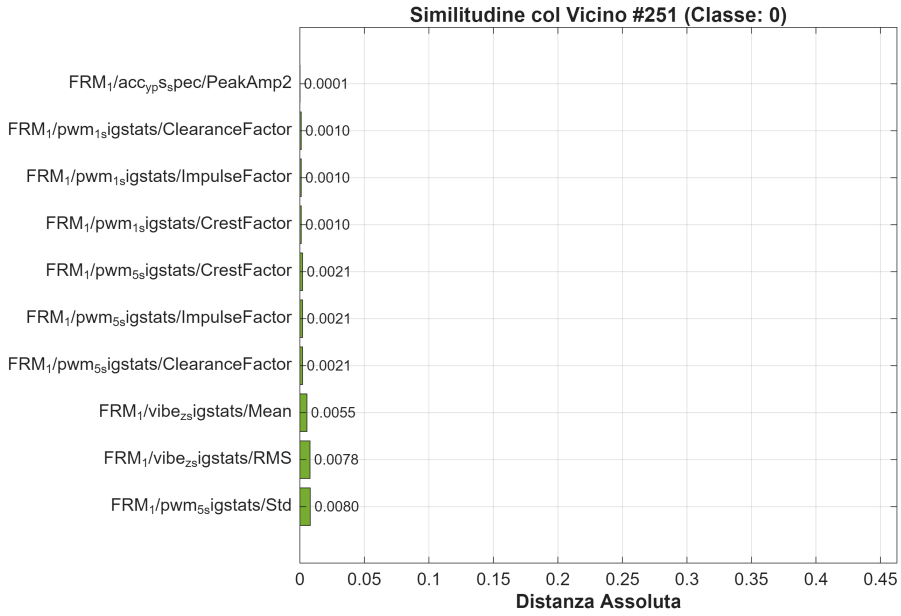


Figura 34: Scostamento assoluto tra il volo analizzato (id 2) e il suo miglior vicino (Stessa Classe)

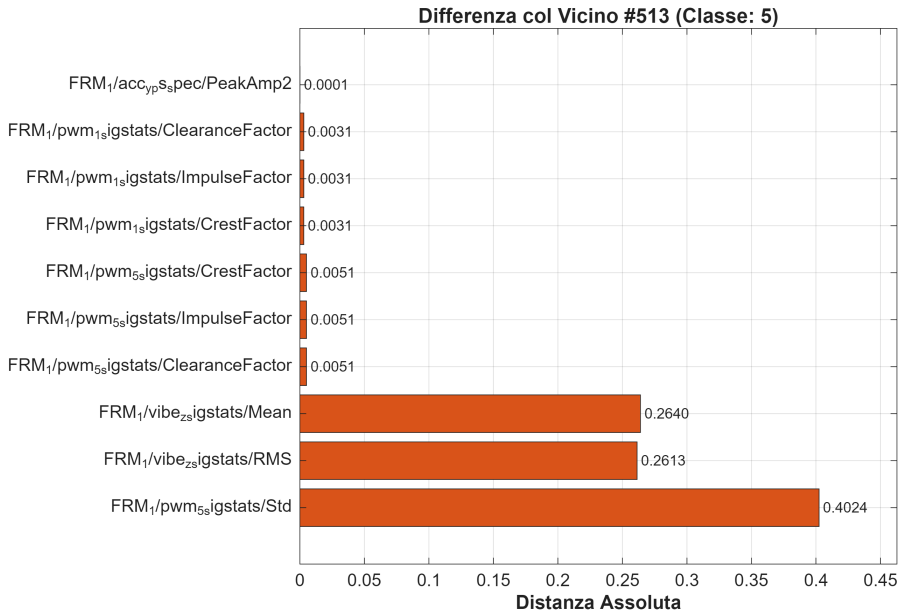


Figura 35: Scostamento assoluto tra il volo e il vicino più prossimo di Classe Diversa

La lettura congiunta dei due grafici permette quindi di isolare, da un lato, le feature che rendono il volo di test "simile" ai campioni della sua classe (basso scostamento Intra-Classe) e, dall'altro, le feature che lo separano dai campioni delle classi alternative (alto scostamento Inter-Classe). In questo modo il KNN, pur essendo un modello privo di confini decisionali espliciti, risulta comunque interpretabile: la classificazione può essere giustificata indicando quali grandezze fisiche rendono il volo analizzato vicino ai sani e distante dai guasti.

7 Conclusioni

Il lavoro svolto ha dimostrato la fattibilità di un sistema di diagnostica per il rilevamento dell'usura delle pale di un drone esarotore a partire dai soli log di volo acquisiti a bordo. L'intera pipeline, dalla sincronizzazione dei segnali eterogenei di ArduPilot alla costruzione della *timetable* e all'estrazione delle feature nel dominio del tempo e della frequenza, ha permesso di ottenere un dataset strutturato e informativamente ricco, in grado di catturare le modifiche dinamiche indotte dal danneggiamento delle pale nelle diverse condizioni operative (pala sana, usura al 5%, usura al 10%). L'impiego combinato di due tecniche di feature selection, One-Way ANOVA e Kruskal-Wallis, ha evidenziato come un approccio non parametrico risulti in molti casi più robusto rispetto alle assunzioni di gaussianità, garantendo prestazioni medie paragonabili o superiori e, soprattutto, una minore variabilità sulle diverse partizioni del dataset.

Dal punto di vista classificativo, tutti gli algoritmi analizzati hanno raggiunto livelli di accuratezza elevati nello scenario binario di *fault detection*, superando il 91% e arrivando, per Random Forest e Neural Network, fino a circa il 94.5%, con una deviazione standard contenuta. Nel caso più impegnativo di classificazione a tre classi (0, 5, 10), i modelli più complessi (Reti Neurali, SVM e Random Forest) hanno mantenuto prestazioni di circa il $91-92\%$, mostrando una buona capacità di discriminare non solo la presenza del guasto ma anche la sua severità, mentre l'albero decisionale singolo ha evidenziato maggiori limiti nel definire confini decisionali sufficientemente articolati.

Un contributo rilevante del progetto riguarda anche l'aspetto di *explainability*: tecniche quali la *Permutation Feature Importance* per SVM, l'analisi dell'*OOB importance* per Random Forest, il modello surrogato basato su albero decisionale per le Neural Network e lo studio locale dei vicini nel KNN hanno permesso di collegare le decisioni dei modelli a specifiche variabili fisicamente interpretabili, quali i livelli di vibrazione (RMS, Peak Value) e i comandi PWM di controllo. Questo elemento risulta fondamentale in un contesto manutentivo, in cui non è sufficiente individuare il guasto, ma è necessario comprendere quali grandezze ne guidino la diagnosi per supportare decisioni operative consapevoli.

Nel complesso, il lavoro conferma come l'analisi avanzata dei log di volo, opportunamente pre-processati ed elaborati mediante tecniche di feature engineering e machine learning, possa costituire una base solida per lo sviluppo di sistemi di manutenzione predittiva su piattaforme UAV.